

Title to be revisited

Nachtrab Sean

SINF - Computer science
Medical informatics

Supervisors:

Prof. Greet Kerckhofs
Prof. Sébastien Jodogne

Readers:

Isabelle Gennart
Juliette Vanderhaeghen

© This template was adapted by the author from the *Typst KUL template*

Without written permission of the supervisors and the author it is forbidden to reproduce or adapt in any form or by any means any part of this publication.

A written permission of the supervisors is also required to use the methods, products, schematics and programmes described in this work for industrial or commercial use, and for submitting this publication in scientific contests.

Preface

This thesis was launched at the initiative of the writer wishing, in the context of a degree in medical informatics, to work on a problem at the crossroads of two fields, and as a result has two supervisors. It made use of data collected during a previous thesis collected before certain methodological improvements were implemented.

The developed plugin is available at *TODO upload to UCL git*, with the written report code available on github

The Generative AI tool Piccolo was used during the writing of this thesis for feedback and writing modifications. A total use of xxx credits were consumed as detailed further in Appendix A.4. Claude Code was used for support during certain sections of software development, mainly pertaining to the initial setup and 3DSlicer point placement interaction.

Nachtrab Sean

Contents

Preface	i
Abstract	iv
List of Figures	v
List of Abbreviations and Symbols	viii
1 Introduction	1
1.1 Biological motivation	1
1.2 Obtaining information from imaging	3
2 State of the art	5
2.1 Software and Software licenses in Research	5
2.2 Tissue imaging	7
2.3 Vascular Segmentation	10
3 Problem statement	21
3.1 Goal	21
3.2 Objectives	21
3.3 Scope	22
4 Methodology	23
4.1 Tooling: 3D Slicer plugin	23
4.2 User interaction model	25
4.3 Combining algorithms through probability maps	26
4.4 Pipeline components	27
4.5 Ablation study: measuring component contribution	31
4.6 A simplified, scalable pipeline	33
5 Results	35
5.1 Data annotation for evaluation	35
5.2 Performance results	36
6 Conclusion and future perspectives	37
A Appendices	41
A.1 List of software for working with 3D data	41
A.2 CECT dataset performance challenges (TODO: revisit, this was excised from the main text)	42
A.3 Environmental and CO2 impact of the thesis (TODO: revisit)	44

A.4 Use of LLMs and AIs	45
A.5 Development computer specifications	45
B Extra figures (TODO: revisit)	47
Bibliography	51

Abstract

TODO

List of Figures

1.1	Slices illustrating the principal challenges: (a) A large high contrast vessel of approximately 14 voxels. (b) A large high-intensity region not corresponding to vasculature, illustrating staining artefacts. (c) Upper, left: many small vessels in cross-section, ranging from 2 to 8 voxels in diameter, where the partial-volume effect is present, as well as compression artifacts. (d) Right hand side: the “shell effect”, a high-intensity boundary surrounding the tumor caused by the diffusion of the contrast agent. (e, f) Two slices from the same volume, separated by 5 voxels along the z-axis. The vessel indicated appears discontinuous in (e) but continuous in (f), highlighting the relevance of 3D methods.	2
1.1	(a) Large vessel	2
1.1	(b) Staining artefact	2
1.1	(c) Small vessels in cross-section	2
1.1	(d) Shell effect	2
1.1	(e) Slice z	2
1.1	(f) Slice $z + 5$	2
2.1	Top: Grey values of a reliable scan (Red) and unreliable scan (Blue). An ideal scan would be approximately flat within the tumor. Bottom Left: CA-RL-L, considered previously unreliable. Bottom Right: CA-RU-R, considered reliable.	9
2.2	The Frangi vesselness filter: Figure dapted from [1]. Left: The second-order derivative of a Gaussian kernel G''_{σ} probes inside-versus-outside contrast over the spatial range $(-\sigma, \sigma)$; here $\sigma = 1$. Convolving an image with this kernel measures for each point how strongly intensity curves over the scale <i>sigma</i> : high response means a detected peak or valley of width comparable to <i>sigma</i> - “matching” vessels. Right: At each voxel, the eigen-decomposition of the Hessian matrix yields three principal directions of curvature $\lambda_1, \lambda_2, \lambda_3$. For a tubular structure, the eigenvalue along the vessel axis (λ_1) has small magnitude (intensity is roughly constant along the vessel) while the two perpendicular eigenvalues (λ_2, λ_3) have large magnitude and the same sign (reflecting the drop in intensity in either direction perpendicular to the vessel) this constitutes the geometric signature Frangi’s filter exploits and are combined to form the Frangi vesselness score.	11

2.3	Vascular segmentation methods. x-axis: training-data needed. y-axis: level of encoding of vessel topology. Classical methods cluster on the left, requiring no training data, but vary in the priors they encode: Otsu encodes none, Frangi encodes local tubularity but not connectivity, minimal-path methods enforce connectivity by construction at the cost of user-provided seeds. Data-driven methods (U-Net, V-Net) cluster on the right with high data needs and lack topological awareness when trained with voxel-wise losses. <i>Hybrid approaches</i> as used in this thesis combine vesselness inputs with a learned correction stage.	14
4.1	Structure for module implementation from [2]. The module being implemented must create a widget and a piece of logic that communicate via an MRML node. This motivates our separation of the code into two: a main plugin and a library of implemented logic	24
4.2	The QT user interface designer, as seen during the development of the plugin. Key elements of this UI development method are the drag and drop nature as well as limited styling options.	25
4.3	View of the seed annotation and vessel size definition panes. The user may press <i>add</i> and click on the locations in any of the right hand panes to place one or more points. Annotations can be imported or exported.	26
4.4	Visualization of the vesselness probability map as defined by the Frangi feature. Intensity corresponds to vesselness probability. Also visible, user placed points used to guide the algorithms.	27
4.5	The shortcoming of threshold based segmentation visualized, with a “shell” of high valued outside being included whe the threshold accepts the vessel segment.	28
4.6	Structurally aware reconnection: Red: vessels as identified by other steps. Yellow: Bridges between tubular endpoints. ¹	29
4.7	Visualization of the segmentation following the multi stage pipeline during development. Successful continuous vessel segmentation and few small noisy sections. Performance, as measured using user annotated points, of 26/27 vessels and 16/16 background points correctly classified. Also visible: Blob-like artifacts from flood fill weight not being suppressed by Frangi, alongside disconnections in due to larger low signal areas.	30
4.8	RAM utilization before & during segmentation: baseline after loading dataset, below 24GB utilization. During processing 100% of the RAM and swap are used, meaning that this dataset already represented the approximate upper bound to dataset size able to be processed at this point.	31
5.1	Grid subsample of a tumor for annotation. TODO: make this clearer and detail data distribution	35
1.1	TODO: This is a placeholder. TODO: Add horizontal lines for the RAM of computers.	

¹3DSlicer smooths visualizations in 3D without combining different classes. The final segmentaton here is unifrom and continuous.

Visualization of the raw dataset sizes, obtained by multiplying width, height and depth by 8 bits per pixel	43
1.2 Top: CA-LL-L2 RAM utilization during segmentation: baseline after loading dataset, below 24GB utilization, during processing 100% RAM and swap are used, Bottom: CB-LURU-R showing full RAM and SWAP utilization when loaded (no inference)	44
2.1 Demonstration of one of the failing plugins and the lack of user feedback on what went wrong	47
2.2 Areas of CA-LL-R where vessels are detected because of the black holes around it	48
2.3 Import process	48
2.4 RAM and CPU use during a run of the initial pipeline on CA-LL-R showing the efficiency issues that Python and 3D Slicer cause	49
2.5 RAM use loading a large dataset, showcasing the issue with 3D Slicers method of loading data into memory	49

List of Abbreviations and Symbols

Abbreviations

TODO: extending this

CT	Computed tomography, method for 3D imaging
Micro-CT	High resolution CT
CECT	Contrast-Enhanced Micro-CT
CESA	Contrast-Enhancing Staining agent
CCA	Casting Contrast Agent
GT	Ground Truth

Chapter 1

Introduction

Biological tissue function depends strongly on spatial organization and composition. This is especially true for vascular networks, spread inside all other tissues, for which the geometry of branching, vessel diameters and connectivity define functionality. In order to obtain an understanding of a tissue, 2D histology is the gold standard, but its limitations in capturing three dimensional structure have driven the adoption of 3D imaging techniques, such as micro-computed tomography (Micro-CT) and its contrast-enhanced variants aiming to improve tissue separation. These methods, and their high resolution subtypes, generate orders of magnitude more data than traditional microscopy and pose new challenges in information extraction and processing.

In the context of research on angiogenesis - the process of vascular growth exploited by tumors that antiangiogenic drugs aim to disrupt - quantifying changes in vascular structure is essential to evaluating treatment efficacy. The dataset motivating this work consists of contrast-enhanced Micro-CT scans of murine tumors collected to assess the antiangiogenic drug Pazopanib, whose mechanism of action is expected to manifest as measurable changes in vasculature.

However, existing tools for analyzing Micro-CT data are dominated by proprietary paid software and the available open-source alternatives lack robust solutions to extract small-scale vasculature with its challenging specificities: low contrast, diffusion gradients, and vessels only a few voxels across. As a result, the vasculature in the provided datasets has resisted prior analysis attempts.

To address this, a pipeline for microvasculature extraction from Micro-CT data is developed in the form of a 3D Slicer plugin, an open-source tool widely used in medical imaging. Subjective hyperparameter tuning is replaced by parameters derived from sparse user input, making the pipeline both principled and adaptable to new datasets without retraining or expert configuration. Usability is prioritized to bridge the divide between computer vision research and biological use.

1.1 Biological motivation

Tissue samples are collected and imaged in order to gain an understanding of their structure and composition, called histology or histopathology in the case of diseased tissue. Classically this is done ex-vivo in 2D, with sections analyzed manually under a microscope by a human, the gold standard for tissue analysis [3]. However the technique has a fundamental limitation:

information is not captured at the same granularity along all three axes. Methods exist to stack slices and reconstruct a 3D volume @ [4], but axial resolution is limited by section thickness, and dead space between slices leaves regions where no information is acquired [5] and the cutting process itself induces changes in tissue structure [6].

Evaluating the antiangiogenic drug Pazopanib, a tyrosine kinase inhibitor influencing the formation of new blood vessels, requires extraction and quantification of the tumor vasculature. Vascular networks are inherently 3D and span entire tissues, having abstract parameters relevant to understanding drug administration response. This 3D nature, combined with the limitations of slice-based methods, motivates high-resolution 3D imaging: for the dataset used in this work, contrast-enhanced Micro-CT scans were acquired using an agent that increases the attenuation of the small blood vessels.

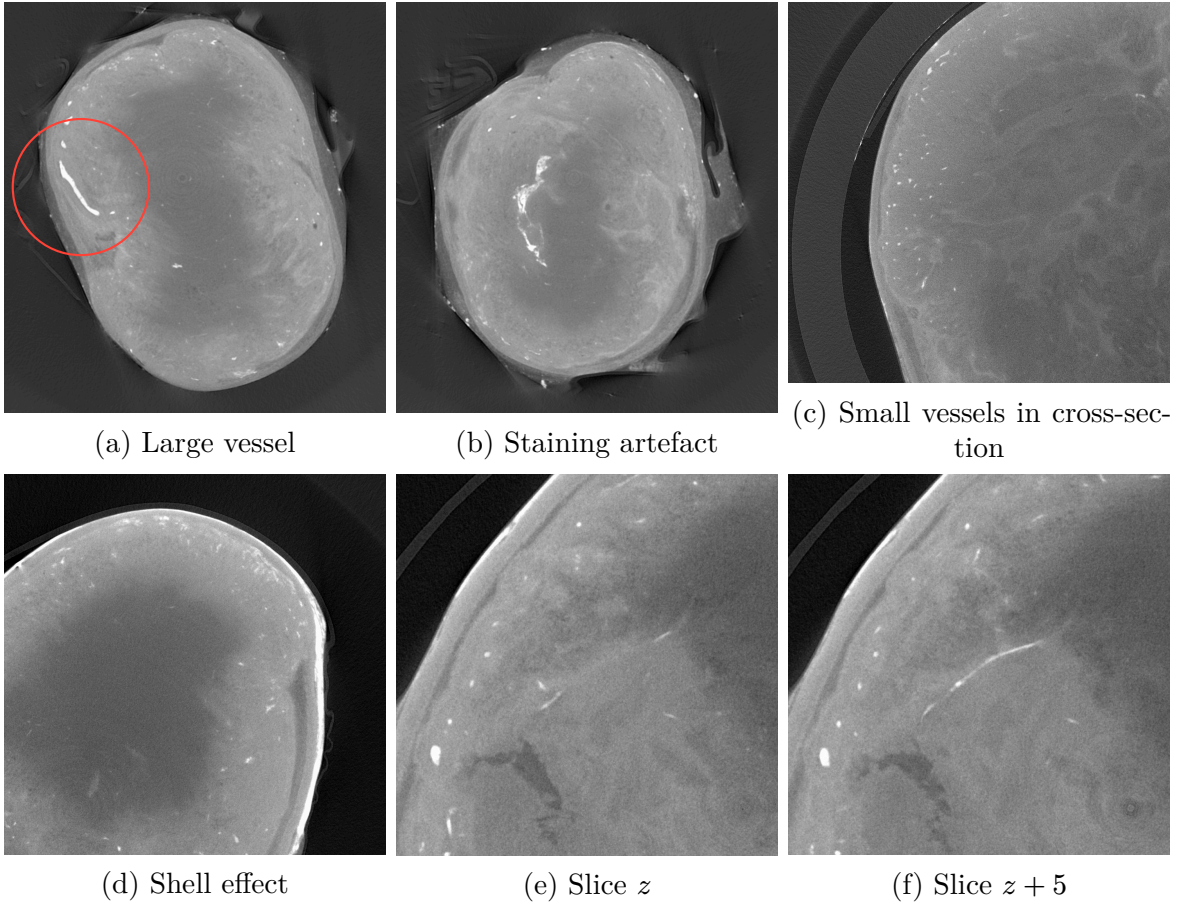


Figure 1.1: Slices illustrating the principal challenges: **(a)** A large high contrast vessel of approximately 14 voxels. **(b)** A large high-intensity region not corresponding to vasculature, illustrating staining artefacts. **(c)** Upper, left: many small vessels in cross-section, ranging from 2 to 8 voxels in diameter, where the partial-volume effect is present, as well as compression artifacts. **(d)** Right hand side: the “shell effect”, a high-intensity boundary surrounding the tumor caused by the diffusion of the contrast agent. **(e, f)** Two slices from the same volume, separated by 5 voxels along the z -axis. The vessel indicated appears discontinuous in (e) but continuous in (f), highlighting the relevance of 3D methods.

1.2 Obtaining information from imaging

Analyzing tissue requires imaging it at a sufficient resolution to extract the structure of interest, and 3D imaging modalities leveraging tomography such as Micro-CT and Micro-MRI now make this possible at micrometer resolution. However moving from 2D to 3D imaging carries with it challenges: a single high-resolution Micro-CT scan produces thousands of slices and orders of magnitude more data than a comparable 2D acquisition, meaning that analysis as with 2D microscopy at this scale is impractical. Micro-CT data itself is also highly variable: scans differ on the same machine due changes in imaged tissue, contrast agents and protocols used to make structures of interest visible, and further variation arises across machines and due to researchers' choices of acquisition parameters.

When faced with complex data the paths for researchers are twofold: either *manual annotation* of the data, or *utilization of algorithms* to aid in the separation of their structures of interest. Manual annotation suffers from high subjectivity, strong variance between annotators [7] and requires time and expert annotators. Additionally, annotation natively in 3D is challenging: it is difficult to properly consider all three axes simultaneously, hindering work on small 3D structures like vasculature.

When using algorithms for extraction, users are pulled towards user friendly and simple methods: (1) generalist methods such as thresholding, easily understandable and widely available, but performing poorly on data that cannot be separated by intensity alone, and (2) a wide spectrum of techniques that carry a richer prior or set of priors about the target structure. For vessel analysis, simple methods are easy to apply but result in discontinuities in blood vessels, artefacts, and don't generalize across samples, as well as require the selection of a fixed threshold per sample, a step that researchers reported performing by eye and using prior experience rather than by any principled manner, as learned from user interviews.

The uptake and adoption of advanced techniques remains limited outside of the walls of computer science labs due to the barrier to entry associated with using a new tool or method being too high: proprietary paid tools are used, research software is often unmaintained or poorly documented and algorithms are challenging to run on different machines. Modern deep learning based algorithms add even more barriers: dedicated GPUs are typically required [8], input data must be converted to specific formats, and out-of-distribution data often demands retraining, requiring data and computer science knowledge.

Contribution and scope

Work will be carried out to create an open-source and data driven small blood vessel extraction method for obtaining segmented blood vessels, with the goal of enabling downstream extraction of clinically relevant vasculature characteristics. This goal is defined by the usecase of a dataset of Contrast-Enhanced Micro-CT data acquired to examine the use of Pazopanib. The pipeline is created as an easily installable plugin for 3D Slicer, an open-source and free base software, that ingests and outputs data in portable data formats familiar to researchers, to ensure re-usability and compatibility with existing downstream software. The plugin implements proven algorithms with sparse user input leveraged to set principled hyperparameters enabling easy re-use without requiring expertise in computer

science. Implemented methods are tested for robustness by examining performance on data previously considered non segmentable due to non uniform contrast agent staining.

All code written in the implementation of this thesis, the source code of this document, and the working notes during writing, are made open-source under the MIT license (for code) and labeled under the creative commons as CC0 - No Rights Reserved (for creative works - the writing).

Chapter 2

State of the art

2.1 Software and Software licenses in Research

Software licenses are closely linked with the monetary and scientific costs of use: they don't only govern the terms of software use but also of modification, influencing the ability to build upon existing work. They broadly fall into two categories: proprietary (closed source) and open source. Proprietary software restricts access to its source code and is generally distributed under a paid license, with certain exceptions such as with Dragonfly3D's FreED license. Open source software makes its source code publicly available and is as a result free of charge. In a research context, the distinction matters beyond cost: open-source code is by nature extensible, with prior work directly inspectable and modifiable, while proprietary licenses can enable the owner of the original software to control the distribution and use of extensions: Dragonfly3D does not explicitly allow sharing extensions². 3D Slicer's license, by contrast, permits modification and redistribution: see 3D Slicer's license page. Open-source licenses themselves vary in how they govern downstream use, from permissive (MIT, BSD) to copyleft (GPLv3), with implications for how derivative work must be redistributed.

2.1.1 Software for 3D analysis

Researchers at the UCLouvain faculty IMMC (Institute of Mechanics, Materials, and Civil Engineering) use a variety of software to process 3D Micro-CT data: closed source in the form of Avizo and CTan, "free-for-academics" with Dragonfly3D, and previously used open-source in the form of ImageJ/FIJI. A full list of available solutions is visible in Table 2.3. Standalone approaches exist, such as DeepVesselNet [10] and SPROUT [11] but do not come packaged as a software with user interface.

For this thesis, software was required to meet the following three hard requirements: **(1)** Import a 3D scan from individual 2D slices in standard formats, **(2)** Export data to non-proprietary formats, and **(3)** Allow coded plugins/code extensions. Some practical considerations were also taken into account: availability of dedicated support and documen-

²“[The user] shall not distribute or transfer the Software or Improvements [...], without prior written permission [from Dragonfly3D]” [9]

tation, active development, prior use within the lab and the broader research community, and ease of installation for non-technical users.

Each candidate platform was tested for the practicality of implementing a plugin within its extension framework. Avizo and Dragonfly3D met the hard requirements but limit plugin redistribution as laid out previously. ImageJ/FIJI has a long history in biological image analysis and a rich plugin ecosystem, including use for vessel extraction [12], but native support for vascular network extraction in 3D is limited, and the plugin development model is less suited to the integrated segmentation workflows this thesis requires.

3D Slicer [13] emerged as the most suitable platform. It is open-source under a permissive BSD-style license, widely adopted in the medical imaging field, and supports 3D segmentation workflows with established plugins relevant to vascular analysis such as the Vascular Modeling Toolkit (VMTK) [14] and R-Vessel-X [15]. Its Extension Manager allows non-technical users to install plugins through a one-click interface without compiling code or managing dependencies, and standard import/export formats including DICOM and NIfTI ensure interoperability with downstream analysis tools.

Problem 1.

Existing tools for 3D analysis are split between proprietary paid software and open-source platforms, neither having bespoke microvascular support. The pipeline must therefore integrate into an open-source software for 3D analysis: 3D Slicer.

2.2 Tissue imaging

The process of imaging is a critical step in obtaining an understanding of a tissue in both clinical and research contexts. 2D histology is considered the gold standard having a long history [3] and producing high resolution images down to 5 μm for light based methods. The diversity in imaging techniques for 2D histology has rapidly increased the resolution, with electron microscopy able to reach in the order of sub nanometer scale [16]. Progress has also increased the dimensionality of imaging, with computation enabling rapid progress in 3D methods using techniques like tomography, where multiple images are taken non-destructively of a target from different angles and assembled, used in-vivo and ex-vivo methods such as computed tomography (CT) and magnetic resonance (MRI).

For tasks such as measuring the impact of a drug on vascularization, quantification is particularly relevant: Understanding the three-dimensional structure of biological tissue is a prerequisite for the analysis of structure-altering drugs such as Pazopanib, where changes in tumor vascularization are inherently spatial and quantifiable across multiple parameters. For such tasks, 2D histology is insufficient: Physical sectioning of the sample is destructive, is not orientation-agnostic, and introduces deformation artifacts that are difficult to compensate for even with embedding techniques [17], additionally samples may undergo structural changes over time during preparation: they dry out, and certain elements oxidize, although techniques exist to mitigate this [5].

To achieve 3D imaging from 2D histology, 2D slices are stacked across a virtual axis [6]. The resolution achievable when stacking 2D slices to reconstruct a virtual third dimension is limited by slice thickness and inter-slice registration errors. Certain optical techniques making use of slices allow partial recovery of 3D information from 2D acquisitions, such as confocal microscopy, light sheet microscopy, and optical coherence tomography [18], they are limited in sample penetration depth and volume. When the target structure to be imaged and understood does not follow a single preferred axis, as is fundamentally the case for vascular networks and especially those of tumors, the limitations of slice-based histology make proper reliable quantification of blood vessel parameters impossible, and the relevance of 3D methods like MRI and CT clear.

2.2.1 3D imaging techniques for histology

Non-destructive 3D volumetric imaging methods overcome many of the limitations of 2D slice based histology by collecting data uniformly across dimensions, enabling virtual slicing across any plane without requiring physically sectioning the sample. Techniques for 3D imaging range from the aforementioned microscopy techniques using visible or near infrared light, to techniques like MRI that utilize magnetic fields or X-Ray imaging in the form of X-ray microfocus computed tomography (CT). These techniques allow the collection of qualitative and quantitative 3D microstructural data of tissues and their constituents: analysis is not restricted to a single orientation and does not require sample destruction. They also have associated high resolution variants allowing micrometer level imaging: Micro-MRI has an inherently high-contrast for soft-tissue and is able to reach 20 μm [19]; Micro-CT is commonly used at 6 μm , as in the data used in this document and resolutions down to 1 μm

are possible, but the technique suffers from low contrast when imaging soft tissue, when extra techniques are not used [3]. These techniques are the most relevant for ex-vivo tissue histology at the scale of small animal models, due to their small maximum capture volume.

2.2.2 Contrast-enhanced micro-CT

As mentionned, standard Micro-CT suffers from low contrast between soft-tissue making the visualization of blood vessels difficult. To improve contrast two common approaches exist: changes to imaging methodology, and modifications to the sample. Imaging techniques such as phase-contrast Micro-CT (PC-CT) utilize the refractive properties of the X-rays rather than its absorption alone, enhance soft tissue edge detection at the cost of higher complexity and long acquisition times. Tissue modifying techniques that aim to increase contrast include the use of various Contrast Enhancing agents (CESAs): Casting contrast agents (CCA) are perfused into vasculature but pressure must be controlled carefully [20] and is not applicable to our usecase due to the size of vessels to be observed. Diffusion contrast-enhancing agents reach the structure of interest and bind to it based on diffusion through the structure to be imaged, termed CECT. For the tumor vascularization studied in this thesis, ex-vivo tissue binding CESAs are used, namely Hafnium Wells-Dawson Polyoxometalates (Hf-WD POM) due to its nondestructive nature and low tissue shrinkage during incubation after excision. Techniques exist that can be used in combination with CESAs: contrast can be increased using freezing to increase contrast Cryogenic contrast-enhanced microCT (cryo-CECT) which preserves tissue microstructure with reduced deformation [18].

Tumors also present a particularity in that they are frequently partially or entirely devoid of residual hemoglobin, preventing or reducing the action of CESAs and thereby reducing contrast and introducing discontinuities. Diffusion based contrast-enhancing agents also bring an additional disadvantage as seen in the data provided for this thesis, visible in Figure 2.1: the diffusion of CE agents throughout the tissue happens from the outside in, resulting in a gradient of the amount of agent and as a result a gradient in contrast, as opposed to perfusion CESAs that follow the blood vessels.

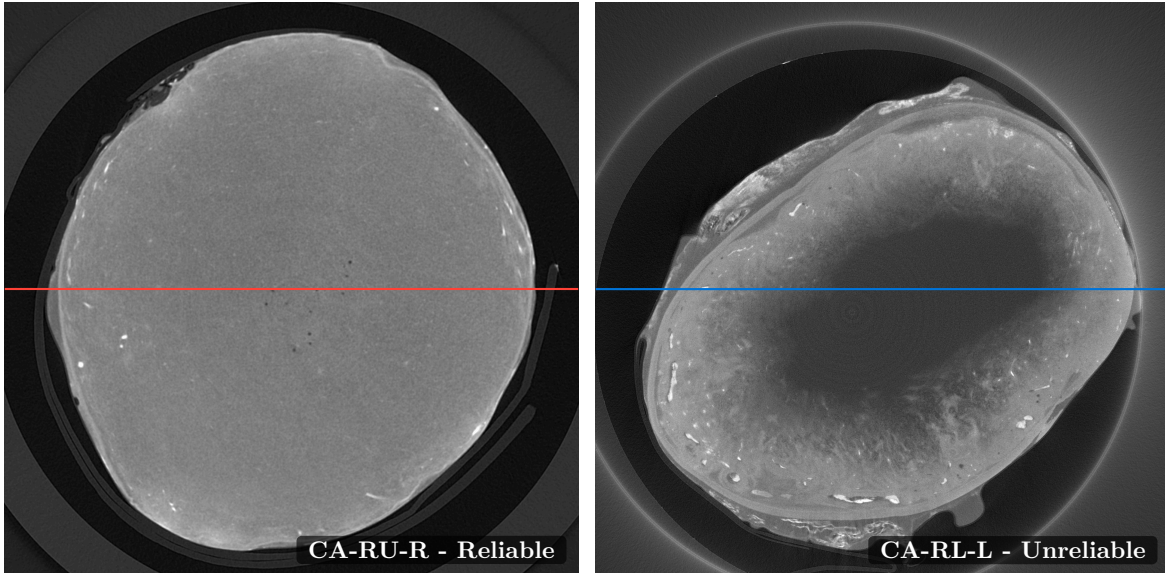
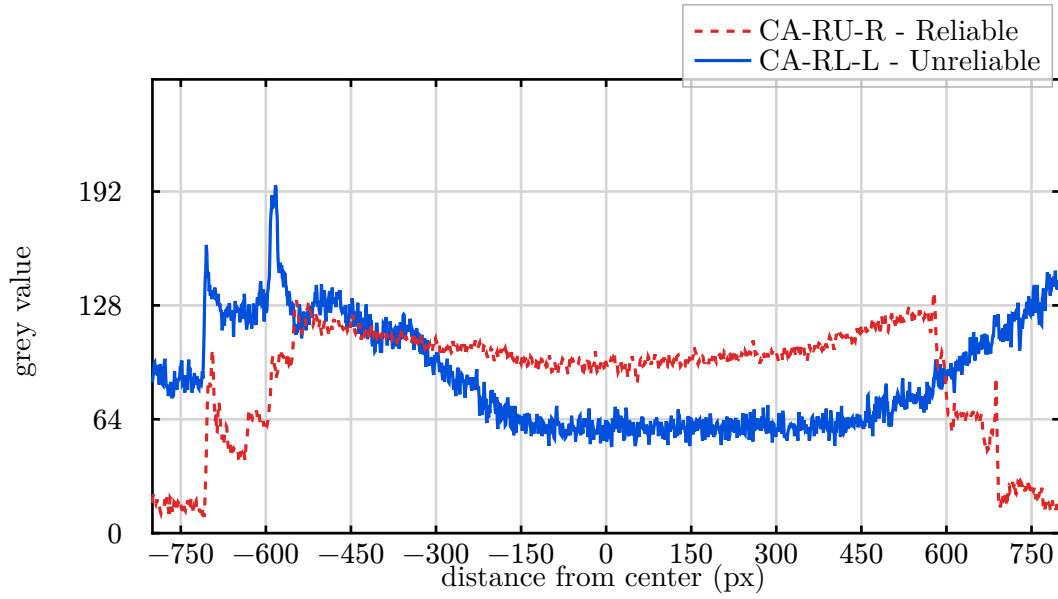


Figure 2.1: **Top:** Grey values of a reliable scan (Red) and unreliable scan (Blue). An ideal scan would be approximately flat within the tumor.

Bottom Left: CA-RL-L, considered previously unreliable. **Bottom Right:** CA-RU-R, considered reliable.

Problem 2.

Our Micro-CT CECT data presents non-uniform contrast gradients across samples in addition to intra-dataset variability. The pipeline must therefore be robust to varying contrast and work on all dataset samples.

2.3 Vascular Segmentation

The purpose of imaging a tissue is to generate an image containing sufficient data to enable useful information to be extracted. This goal can be seen in the progress of imaging techniques: the improvements in the *acquisition process* laid out in the previous section, to make the data of interest more salient (using agents to enhance the contrast) or to make the data more granular (increasing resolution). The *information extraction* process has also seen improvements over time, with analysis evolving from human to computerized through different paradigms: classical thresholding and region growing, geometrically motivated filters and path-based methods to data-driven learning approaches.

Vasculature extraction from 3D tomographic data is a longstanding challenge in computer science, predating deep learning as reviewed in [21]. The 3D data of this thesis carries a unique set of challenges specific to modern high resolution CT: the vessel diversity ranges in the supplied data from singular voxels ($6\mu\text{m}$) to tens of voxels ($100+\mu\text{m}$) across, corroborated in [22]. The vasculature is diverse in shape and branching structure, with variable grey levels and most critically disconnections of variable size, presenting however a distinctive and consistent set of geometric priors.

The priors of vessels, namely that they are tubular, connected, branching structures and contain blood, provides a distinct imaging profile against surrounding tissue when combined with a CESA. They are what subsequent segmentation methods either exploit explicitly through hand crafted classical filters or learn implicitly in data-driven methods.

2.3.1 Classical segmentation

Intensity-based methods

In its simplest form, intensity-based thresholding methods such as Otsu [23] select an optimal global intensity threshold by maximizing inter-class variance across the image histogram. Such methods are computationally inexpensive and interpretable, but are sensitive to noise, imaging artifacts, and intensity inhomogeneities, as they optimize for a numerical goal. Gradient-based and global-threshold methods are a natural first approach to CECT data, since contrast agents are intended precisely to increase the salience of the structures of interest. However, methods that reason globally over the image can fail in practice due to diffusion gradients Chapter 2.2.2 and acquisition noise as mentioned previously.

Region-growing methods [24] extend thresholding by incorporating spatial connectivity: starting from one or more seed points, labeled regions are iteratively expanded according to local intensity criteria, such as in flood-fill, and generalized further by methods such as watershed [25], which treats the image as a topographic surface. Incorporating this spatial connectivity prior makes them more robust to global intensity variation, but remain susceptible to over- or under-segmentation in complex structures: they fail to encode the more specific vascular priors, producing fragmented segmentations in the low-CESA, weak-signal regions characteristic of diffusion-CECT data.

Geometrically motivated methods

Beyond intensity-based methods, algorithms integrating local geometric priors exist such as Gabor filters or Hessian-based filters [26] where the local second-order structure is analyzed to detect tubular shapes, characterized for vessel detection by Frangi’s multiscale vessel enhancement filter [1], with extensions such as [27] and many other methods being created since. Although many improvements to Frangi exist, with a selection of 6 methods compared in [28], Frangi remains highly competitive, being shown to continuously offer the highest true positive to false positive rates.

These second-order methods utilize the eigenvalues of the Hessian matrix of local image intensities (capturing how rapidly intensity changes are themselves changing - the local curvature of the intensity surface) at multiple Gaussian scales (corresponding to multiple candidate “tube” sizes) to produce a vesselness score for each voxel, modeling the prior of blood vessels by responding to tubular structures while suppressing blob- and plate-like ones. Frangi and related methods offer greater robustness than simple thresholding but depend on the user fine tuning algorithm hyperparameters to optimize performance for a given domain or usecase when working with a tool as in [12]. Additionally the filter’s response degrades at vessel bifurcations, at the endpoints of vessels, and in regions of low contrast. In [27] extensions to the vesselness formulation are proposed that improve responses at junctions and at low-contrast boundaries to increase robustness of the filter.

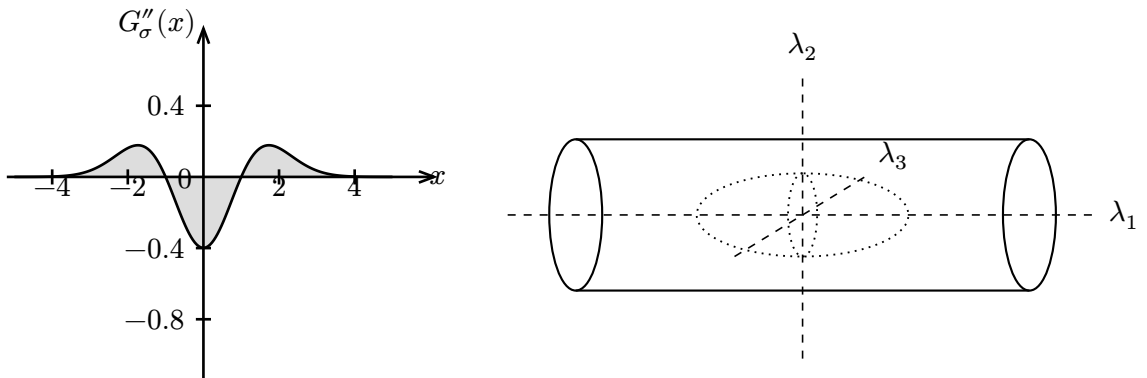


Figure 2.2: The Frangi vesselness filter: Figure dapted from [1].

Left: The second-order derivative of a Gaussian kernel G''_σ probes inside-versus-outside contrast over the spatial range $(-\sigma, \sigma)$; here $\sigma = 1$. Convolving an image with this kernel measures for each point how strongly intensity curves over the scale *sigma*: high response means a detected peak or valley of width comparable to *sigma* - “matching” vessels.

Right: At each voxel, the eigen-decomposition of the Hessian matrix yields three principal directions of curvature $\lambda_1, \lambda_2, \lambda_3$. For a tubular structure, the eigenvalue along the vessel axis (λ_1) has small magnitude (intensity is roughly constant along the vessel) while the two perpendicular eigenvalues (λ_2, λ_3) have large magnitude and the same sign (reflecting the drop in intensity in either direction perpendicular to the vessel) this constitutes the geometric signature Frangi’s filter exploits and are combined to form the Frangi vesselness score.

Although offering high performance, second-order methods are inherently local, failing to incorporate the overarching goal of vasculature extraction. A different class of methods reformulate extraction as a global optimization rather than a local filter response, such as minimal path methods which formulate vessel centerline extraction as an optimization problem. The path of least cost is taken between user-defined endpoints, with cost derived from vesselness or image intensity. In [29] the geometry of vessels is incorporated, adding radius as an additional dimension of the path space. This class of methods is particularly well-suited to expert-in-the-loop approaches where small amounts of data need to be annotated: a user can place seed points that guide the extraction of a complete vascular tree, requiring no training data, and enabling iterative improvement. This method can be seen used for large arteries and airways available in tools such as the Vascular Modeling Toolkit (VMTK) [14]. This strength is also a weakness: the low scalability means that for a densely vascularized tissue, or a tissue with non-uniform characteristics, many seed points can be required which is impractical. Automatic seed point placement is a potential solution although introducing its own tradeoff, moving the optimization and work into the point placement.

2.3.2 Data driven learning methods

Data-driven methods approach extraction from a different direction: rather than hand-engineering features and priors, parameters are learned from labeled examples. Supervised learning algorithms map labeled inputs to outputs with simple methods like k-nearest-neighbours use neighbour voting and more complex gradient-boosted trees combining weak learners [30]. With the paradigm deep learning exploding in popularity with [31], higher dimensionality inputs became analyzable without hand crafted features, and led to fully convolutional encoder-decoder architectures being used in the medical domain such as with U-Net [8], a leap in performance for image segmentation.

U-Net specifically constituted a breakthrough in medical imaging due to its ability to segment large images with high compute and data efficiency, and across multiple scales by introducing skip connections between encoder and decoder pathways. These allow the network to combine low-level spatial detail with high-level semantic context, enabling the segmentation of thin structures that require context such as cells and vasculature. The structure of U-Net can be extended into 3D with different approaches such as [32]. Recently, transformer-based architectures have superseded convolutional networks as a more generalist approach to machine learning, being applied to object detection [33] and being extended to segmentation [34], offering higher performance thanks to a more general computation model. Expert crafted feature extraction is removed, such as in the locality prior of convolutions, enabling the capture of more diverse features and long-range spatial dependencies, at the cost of an increase in required training data, an issue for Micro-CT.

As algorithm complexity and data dimensionality increase, so must the amount of training or example data increase, an issue for convolutional networks and even more so for transformers. This limits the application of deep learning to Micro-CT data, where annotated data is expensive and where inter dataset variance is large, even though attempts exist to palliate the high data requirements by offering self-configuring training frameworks with model weights [35] or by crafting more data-efficient models. Examples also exist of trying

to make use of so called foundation models, based on a large transformer and able to be “prompted” to customize the segmentation to the usecase at hand [36].

Deep learning has been applied to vascular segmentation for blood vessel extraction directly with notable success: DeepVesselNet [10] introduced a family of architectures designed for vessel segmentation, centerline prediction, and bifurcation detection in 3D angiographic data by making explicit use of the structural priors inherent to blood vessels as secondary learning targets.

Finally, hybrid approaches offer the interesting property of combining classical extraction methods with deep learning by leveraging input filters on the image to obtain richer features, such as applying vesselness maps to the image before processing, to integrate the priors of the vesselness filtering explicitly into the algorithm and reduce data needs [37]. This decouples the structural prior, encoded by the filter, from the learning, allowing the learning element to act as a correction or enhancement stage for the shortcomings of classical methods. This use of classical priors as a feature stage with a form of learning as a correction sets the direction of the present work, with the novel addition of sparse user input as a method for achieving training data acquisition.

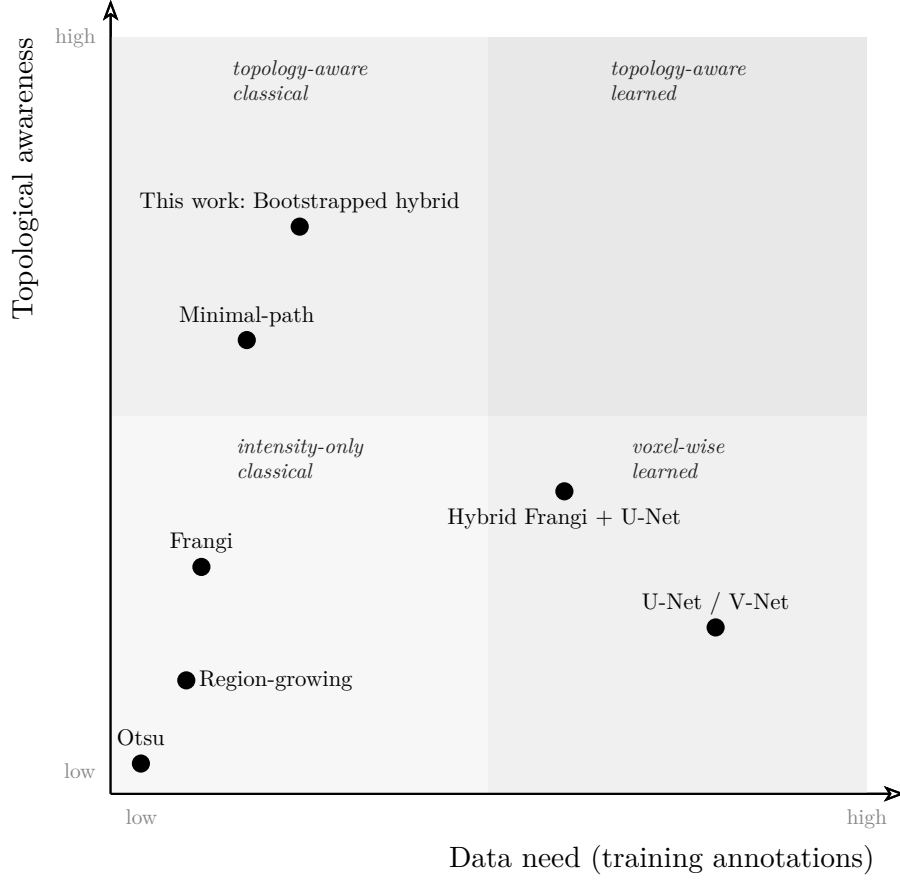


Figure 2.3: Vascular segmentation methods. **x-axis:** training-data needed. **y-axis:** level of encoding of vessel topology. Classical methods cluster on the left, requiring no training data, but vary in the priors they encode: Otsu encodes none, Frangi encodes local tubularity but not connectivity, minimal-path methods enforce connectivity by construction at the cost of user-provided seeds. Data-driven methods (U-Net, V-Net) cluster on the right with high data needs and lack topological awareness when trained with voxel-wise losses. *Hybrid approaches* as used in this thesis combine vesselness inputs with a learned correction stage.

Problem 3.

Segmentation has evolved from classical methods encoding geometric priors toward data-driven methods, requiring annotated training data that is scarce and high variance for CECT micro-CT. Our work should need no training data when used but instead leverage simple user placed points, and automatically adjust the hyperparameters to enable portability across diverse data.

2.3.3 Unreliability of ground truth

Manual annotation of vascular networks in micro-CT images is time intensive and requires expert knowledge. When annotation is carried out, the resulting annotations are known to exhibit significant inter-annotator variability [7]. This variability represents an irreducible uncertainty in the ground truth that places an upper bound on the performance achievable by any supervised method, especially those training on voxel-level data. Work-arounds exist: structural extraction (e.g., skeletonization) may be used on the data first, with the resulting structures used to guide subsequent voxel-level re-annotation, reducing inter-annotator noise. However this means the biases of the skeletonization algorithm are now inherited by the annotations and introduced into models trained on them.

An alternative to real annotated data is the use of simulated and automatically annotated data, using either geometrically driven techniques such as L-Systems, to generate branching tree structures grounded in the physiological laws of arterial branching, reviewed in [38], or neural-network driven systems such as generative adversarial networks (GANs) [39].

When data is available it may be used directly, although models trained on one imaging protocol, contrast agent, sample batch, or tissue type exhibit degraded performance when applied to data from a different distribution: this is termed domain shift, and is particularly acute in imaging methods containing many adjustable parameters and machine hardware specificities like micro-CT imaging. This can require the use of techniques like transfer learning, where a model is first trained on a similar task with large data availability then fine-tuned on a small amount of in-domain data [10]. Annotated data can also be *augmented* to artificially increase the available amount and diversity by applying random degradations, elastic deformations and intensity perturbation while preserving the original annotations.

Problem 4.

Data from different datasets is not necessarily directly usable as a proxy without careful consideration. In domain data manually annotated for performance measurement, especially by a non domain expert, cannot be considered an absolute ground truth. For the purpose of evaluating this work, data will be annotated in distribution, and errors calculated should take into account the potential variability from annotations.

2.3.4 Error calculation

When training a model, or evaluating a method, it is important to be able to measure performance in a repeatable and objective way independent of human feedback. In binary classification, the error rate can be as simple as the proportion of correctly classified examples, however as dimensionality of outputs increases, so does the complexity of evaluation methods.

Loss functions such as cross-entropy used for binary classification calculate the loss on a point-by-point basis, based on the predicted distribution. It can be used for segmentation [8], however vessel segmentation has a severe class-imbalance with vessels being a small minority of voxels. The choice of loss function matters as a result: Pixel-independent losses like cross-entropy treat each prediction as independent, meaning for our biased distribution

there is a prior of predicting background and producing fragmented or incomplete vessel predictions. V-Net [40] extended U-Net into 3D and improved performance by using Dice loss, a method better suited for class-imbalanced settings.

For vasculature specifically, breaks in segmentation can be difficult to reconnect downstream, motivating the creation of custom loss function in [41] called cDice, that integrate the prior of connectedness, and build it into the loss function for predictions. In [42] clinically relevant vascular features are encoded into the loss function. A collection of topology-aware loss functions is available in [43]. Beyond loss functions, the evaluation itself can integrate vessel structure: graph-matching compares predicted and reference vascular trees at the level of branches and bifurcations rather than voxels, enabling metrics on the branch-level [44].

When evaluating a segmentation method, consideration of the downstream analysis of its use to extract relevant features, such as tortuosity and branching ratio, is important to consider. As a result, and driven by the expert-in-the-loop approach, outputs will be evaluated based on point-wise loss of user annotated points, as well as on a manually annotated pixel level baseline relevant for connectivity analysis, an area difficult to capture in the loss.

Problem 5.

Evaluation of a segmentation performance requires integrating the structure of the problem. Our evaluation methodology should make use of prior aware losses when calculating error rates.

2.3.5 Existing pipelines and tools

Several software tools and pipelines have been developed to support vascular segmentation and analysis workflows. Segmentation-focused tools operate on the image itself: the Vascular Modeling Toolkit (VMTK) [14] integrates in 3D Slicer and provides a comprehensive, user-friendly suite for vascular segmentation, centerline extraction, and surface reconstruction, but its methods are oriented toward low resolution, large vessels: arteries, aortas, and major branches. It also relies on seed-based interaction that does not scale to the dense, branching geometry of microvasculature.

ITKTubeTK [45] offers a library of algorithms for tubular-structure segmentation and graph extraction built on the ITK framework, but is packaged as a programming library meaning it requires implementation into a tool or use in code, challenging for non-technical researchers. VesselKnife [46] provides an integrated pipeline specifically targeting vessel segmentation, skeletonization, and graph extraction from Micro-CT data but is a stand-alone application, requiring researchers to leave their existing analysis environment, and is not intended for small vasculature. Tools like SimVascular [47] openrate on the outputs of segmentation and extend vascular extraction into simulation, supporting downstream modeling of blood flow.

The landscape of vascular segmentation tools reflects a tension in bio-informatics, medical informatics and bioimager: general-purpose deep learning segmentation frameworks offer good performance on the datasets they are trained on, but require large annotated datasets and offer limited interpretability or controllability as well as are not baked into tools used

by researchers. Classical model-based methods such as Frangi filtering are more transparent and adjustable, built into existing tools and easy to use, but require manual parameter tuning and struggle with complex vascular geometries. Neither pole serves the specific need addressed by this thesis: microvasculature-scale extraction, integrated into a widely-used segmentation environment, with parameters set by principled means rather than expert tuning. This combination is the niche the present work occupies.

Problem 6.

Existing pipelines focus on large vascularization. This work should fill the niche of small vessel segmentation, where tools are not readily available or tailored to the unique challenges of micro vasculature, and must export in a standard binary segmentation format.

2.3.6 Segmenting large datasets (removable)

I had previously here a SOTA on managing large datasets, as this was a challenge that turned up later and blocked me for a while. Would it be interesting to add?

Data management for Micro-CT scans is a challenge for users: after a scan is completed, they receive data from the CT machine in the form of a collection of 16 bit TIFF files: heavy, with a single 2000x2300 slice at 16bits per pixel weighing **9.2MB**, or as is often the case the data is saved as 3 channels, resulting in 27.6MB, and a whole 2400 slice scan weighing at least **22.1GB**. Scans are then windowed to 8 bit, occasionally with some form of compression, and the empty slices are removed: this generally halves or more the total data amount. This windowing process was documented as being unprincipled: the window was chosen based on the researchers best judgment, and the original uncompressed data discarded.

Furthermore, certain researchers would then carry out a lossy compression of the data in the form of JPEG image slices, as was the case with the data used in this thesis: the total scan weights provided ranged from **0.103** to **13.2GB** (597x698x854 to 3000x3000x2653) and the original lossless data was not preserved, in both cases the windowing and the compression were motivated by data storage cost concerns.

Finally, the provided data was generally given with little or no context: the data was provided in the form of a folder containing images as well as experiments that were run, with no associated dates and without grounding context such as the scan voxel size or parameters of the scanning machine. These issues of dataset size and compression resulted in challenges unforeseen during the literature review which required particular attention.

Methods exist to handle large datasets: the most basic approach is cutting down of full scans into smaller chunks, or subsampling the scans with some form of interpolation. Cutting scans down has the disadvantage of requiring stitching after running algorithms, and if done using 3D slicers' built in slicing, requires the ability to load the full dataset. Subsampling requires the the target structures to be large enough to allow it: subsampling to 1/4 resolution means that any vessel 4 voxels across would be reduced to approximately a single voxel.

Handling large datasets can also be done at the format and loading level: HDF5 is intended to *store* such large multidimensional arrays and efficiently enable loading subsections, however this data storage format is totally incompatible with most medical imaging software, and is not the standard used by CT machines. Standards such as DICOM did not provision for the possibility that data such as those generated by micro-CT may exist in the future in the medical domain, and do not deal well with dynamically loading large datasets from memory. To *operate on* large multidimensional arrays in Python, there exist libraries such as Dask that enable "chunking" of the data to process smaller areas: this could enable improved scaling to larger scans.

These performance concerns highlight a continuous issue encountered during the writing of this thesis: the complexity of methods able to be tested was limited by the choice of software, volume of data and the hardware available. Lab computers available to students have 32GB of ram, less than the computer used for the testing and writing of code, and it was noted by previous students working on MicroCT imaging that they had struggled to run algorithms

across the whole image. In the end, much effort was invested in the research, testing and optimization of the algorithms, and runtime concerns pushed development towards the use of methods implemented in C++ available with Python bindings, such as the SimpleITK Frangi filter used.

Chapter 3

Problem statement

In a diverse software landscape for analysis of 3D data, this work aims to answer the question “*How can an open-source microvasculature extraction pipeline be developed, able to be used by non computer scientists, leveraging classical segmentation methods and sparse user-driven input to obtain useful segmentations on CECT data across a diverse dataset?*” and is defined by three principal constraints:

(i) Data: Contrast-enhanced micro-CT of tumor microvasculature with compression artifacts, intensity gradients and discontinuities, and a highly imbalanced data distribution with small target structures only a few voxels across.

(ii) Segmentation and its two paradigms: *data-driven* methods (machine learning) with potentially high performance but poor generalization and requiring annotated training data that is difficult to generate high-variance for CECT, and *classical* methods that encode vessel priors explicitly but require hyperparameter tuning, and reason locally with poor extrapolation resulting in fragmented segmentations and weaker performance in low-contrast regions.

(iii) Tooling: existing software is focused on low resolution, on human arteries and organs with large vessels and few disconnections. Microvasculature extraction methods are sparse, rarely bespoke or integrated into available tooling

3.1 Goal

This work will develop a 3D Slicer extension for microvasculature segmentation from the provided data of CECT imaged murine tumors, intended for use by non computer scientists. The algorithm will be able to infer vessels on both the data previously considered “reliable” as well as those samples labeled “unreliable”, without requiring annotated training data. The development will integrate user feedback and consider the limitations of 3DSlicer as well as challenges imposed by dataset size.

3.2 Objectives

1. Deliver a user-friendly 3D Slicer extension.

2. Leverage a user-in-the-loop approach for point placement & basic vessel-size context to drive automated parameter selection, replacing manual error-prone hyperparameter tuning.
3. Build the segmentation core on multiple algorithms, combined through a framework that enables per component tuning.
4. Produce a pipeline focusing on performance and simplicity first
5. Export useful segmentations for downstream analysis

3.3 Scope

This work delivers the segmentation pipeline in the form of a plugin and measures results on manually annotated data, with downstream quantitative analysis left to existing tools by exporting binary masks. Deep learning in its most common form is not utilized due to a lack of available reference data, training data and usability concerns. The plugin is intended for use in a research setting and tested on the set of data provided with its known shortcomings.

Chapter 4

Methodology

This chapter documents the development of the 3D Slicer plugin, from the implementation and user interaction, through the method for combining algorithms and the different algorithms explored, and ending in an ablation study to simplify the pipeline and enable usable runtimes and memory use.

4.1 Tooling: 3D Slicer plugin

The research team providing the data actively used three tools: the proprietary Avizo, the free-for-academics Dragonfly3D, and a bare metal approach using python code. For plugin development, Avizo and Dragonfly3D were considered, however documentation was challenging and licensing meant it would not satisfy the goal of an open source tool offering extendability and re-usability. As a result, 3D Slicer was selected.

The 3D Slicer ecosystem is diverse with many existing tools, available plugins with similar goals (vascular extraction) and similar data formats (high resolution CT). These were tested in order to obtain an understanding of potential pitfalls in implementation:

- 3D Slicer hangs when executing algorithms, and this execution happens without communication. The OS will ask the user if they wish to stop the program.
- UI development is restrictive: UI must be in the left corner, and few context menus are available
- Execution in Python is inefficient, and plugins that aimed to use multiple cores called external libraries
- Machine learning extensions did not come pre-packaged with the model weights and often required a GPU and installation of external software
- Errors during installation were not user-friendly enough for a non computer scientist to be able to debug an installation problem
- The 3D Slicer forums contained ample amounts of users having issues with different plugins

Additionally many available plugins were either unmaintained, throwing errors during installation/use or did not successfully produce outputs; this is despite 3D Slicer having two classes of plugins: “official” via the extension manager [48] (further broken down by maintenance type) and user installable or “non official”. To mitigate this potential issue, the plugin is packaged with a known good version of 3DSlicer.

With these issues in mind, development of the plugin began: 3D Slicer is multi platform and built around plugins to carry out tasks, communicating using the Medical Reality Markup Language (MRML) Figure 4.1, where modules read and write to MRML nodes to enable interoperability. These plugins can be implemented in one of three forms: as command line interface (CLI) tools where they may be called as an encapsulated piece of code, the most abstracted way of working with external code or a C++/Scripted (Python) loadable module [2]. The Scripted loadable approach was selected for its use of Python, lowering the barrier to entry, ability to use the UI features of 3D Slicer, and having access to the full slicer API. Work was carried out in Visual Studio Code under Ubuntu 24.04.

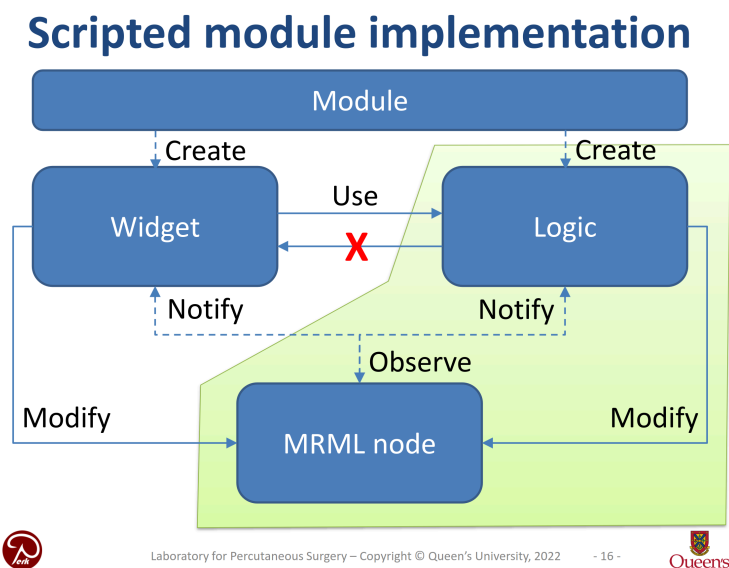


Figure 4.1: Structure for module implementation from [2]. The module being implemented must create a widget and a piece of logic that communicate via an MRML node. This motivates our separation of the code into two: a main plugin and a library of implemented logic

To implement a plugin, the extension wizard³ was used to generate the required extension boilerplate, and the ScriptedLoadableModule was used as a starting point for the implementation code. 3D Slicer implements a user interface development module in QT, enabling near drag and drop creation of UI elements, making it the natural method of developing a seamless UI experience. Although basic, this radically simplifies the process of creating a cross platform UI.

³https://slicer.readthedocs.io/en/latest/user_guide/modules/extensionwizard.html#extension-wizard

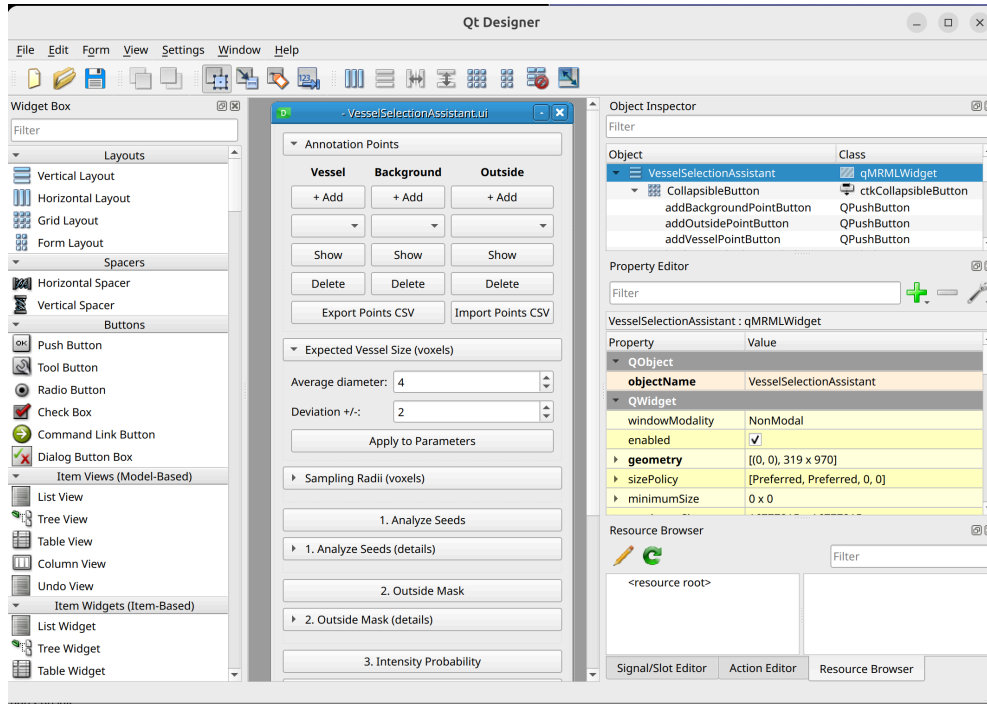


Figure 4.2: The QT user interface designer, as seen during the development of the plugin. Key elements of this UI development method are the drag and drop nature as well as limited styling options.

4.2 User interaction model

Having the user in the loop is essential to software development, and during the thesis multiple consultations were done. During the user interview concerning the current pipeline of data processing, it was noted that decisions such as hyperparameter selection, windowing and grey value were often made without a principled, replicable and data driven decision method. This is problematic for multiple reasons:

1. As the selected values are not data driven, decisions and pipelines cannot easily be replicated, and are more fragile
2. As the user acts as the evaluator to the actions done, they induce bias in the subsequent steps
3. Any user without knowledge of the downstream steps or what to look out for will not be able to make a fully informed decision for the hyperparameters to select, requiring iteration and potentially falling into local optima.

A principled approach was required: in order to select thresholds and evaluate the performance of the algorithm “in the loop”, as well as offer the user immediate feedback, a system of anchor points was implemented. These anchors are the first step in the process, and are obtained by having the user click “add” for any of the three categories, followed by placing the point in any of the 2D windows. These “Vessel”, “Background” and “outside” co-ordinates can be saved and exported, as well as imported in a universal CSV format.

Following the receipt of the data, and from the aforementioned interviews, it was also noted that users often do not take the time to properly name files and folders. Consequently, naming when exporting is automatically constructed from the current data series name and description, as well as the full current date and time. The second key user feedback element implemented to guide downstream steps was the expected vessel size and deviation in voxels. This parameter is easy to measure and provides critical context for automated hyperparameter selection.

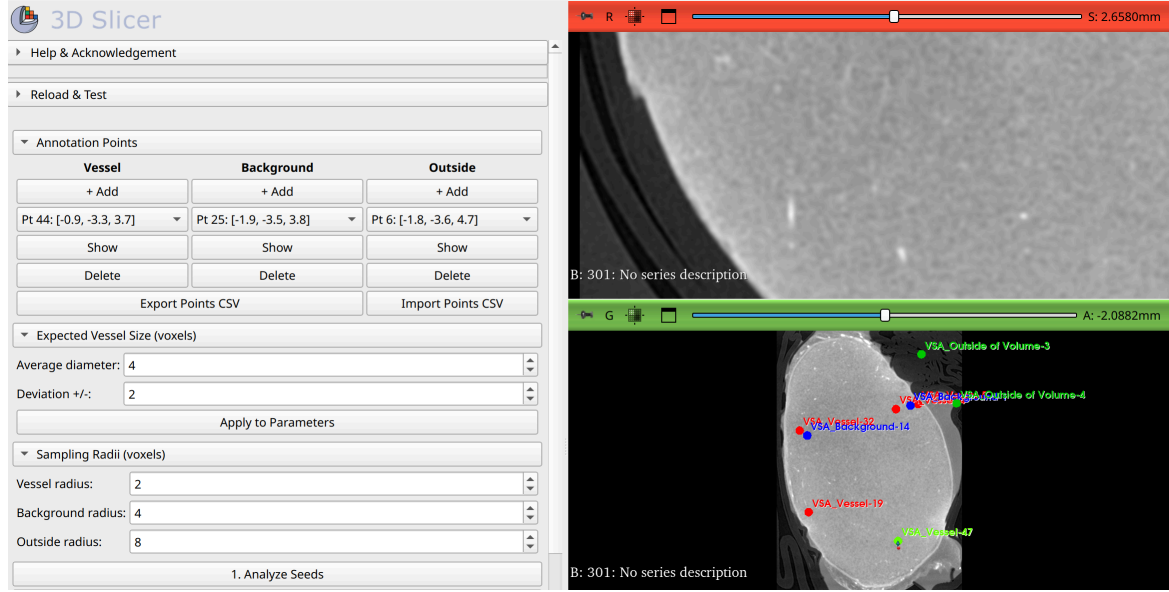


Figure 4.3: View of the seed annotation and vessel size definition panes. The user may press *add* and click on the locations in any of the right hand panes to place one or more points. Annotations can be imported or exported.

4.3 Combining algorithms through probability maps

During the research phase, multiple relevant algorithms for vascular extraction and segmentation were identified. In order to combine the information provided by e.g. thresholding with that of frangi and explicit the process of evidence accumulation, a method called here **probability maps** was implemented where each algorithm or method saves point wise probabilities. These probability maps are the same size as the volume being analyzed, and contain a floating point value for each point [0.0-1.0]. This enabled flexibly weighing and stacking different methods to directly view the segmentation results across different steps in the pipeline. It also enables, by utilizing per method probability maps, for rapidly iterating the weights used to obtain the final segmentation.

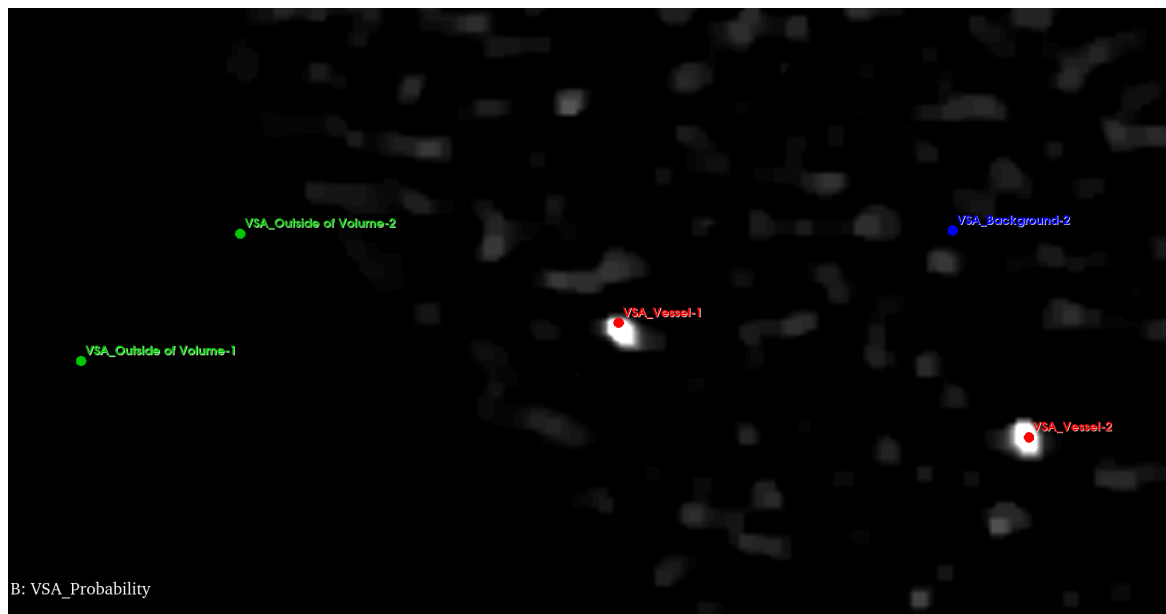


Figure 4.4: Visualization of the vesselness probability map as defined by the Frangi feature. Intensity corresponds to vesselness probability. Also visible, user placed points used to guide the algorithms.

4.4 Pipeline components

To obtain a segmentation on the challenging data provided, no clear single method appeared sufficient. As a result, using techniques that have diverse trade-offs is relevant.

4.4.1 Thresholding (with shell removal)

As noted in Chapter 2.3, the simplest method of segmentation is thresholding. Given the nature of CECT intends to give the structures of interest high grey values, this is an intuitive method with a strong prior. It is however not sufficient alone for three reasons: **(i)** the shell effect in CECT, where the contrast enhancing agent has higher concentrations on the outside or surface of the tumor, results in values that would be segmented as “vessel” even though they are not. **(ii)** grey value gradients appear between the outside and center of the samples and **(iii)** incomplete staining resulting in discontinuous vessels.

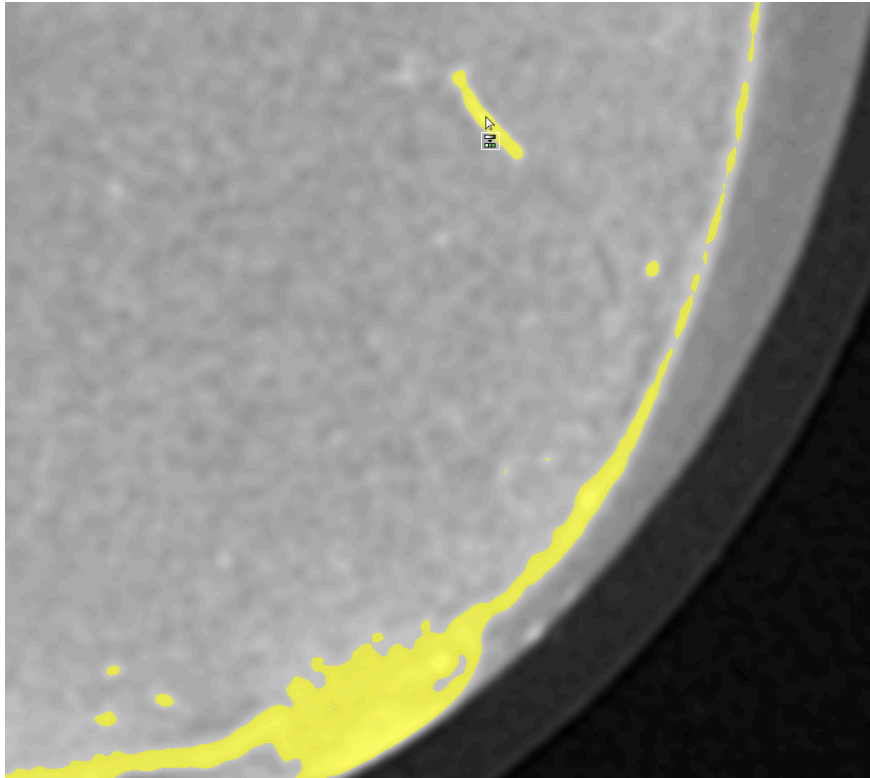


Figure 4.5: The shortcoming of threshold based segmentation visualized, with a “shell” of high valued outside being included whe the threshold accepts the vessel segment.

The shell effect was tackled by fitting a shape to the outside of the sample, however following a user interview demonstrating the feature a critical issue was raised: in tumors, it is common for the outside to contain large and plentiful vascularization. Removal would both bias results as well as reduce the overall performance by preventing these outside vessels from being segmented. Secondly, a threshold does not hold up to the gradients in images that are present which, in prior work carried out on this data, resulted in rejection of samples with a gradient considered too large. Finally, standard thresholding does not do anything to combat the incomplete staining resulting in discontinuous vessels. As a result, thresholding was utilized in the probability map stacking after shell removal, but given a low contribution.

4.4.2 Frangi vesselness

To go beyond the limitations of threshold based segmentation, the generalized C++ implementation of Frangi in SimpleITK was used. The algorithm was applied at multiple scales (different expected vessel sizes) with the outputs collected together into a probability map. Frangi is known for having multiple hyperparameters that require fine tuning: in order to offer the user a simple solution, the parameters for the minimum and maximum vessel scale are pre-defined, and the vessel parameters fixed.

4.4.3 Gap bridging

As other steps had the tendency to result in disconnected regions, as well as given the knowledge that the staining method itself resulted in disconnected regions, a gap bridging step that attempted to link nearby disconnected island was identified as relevant to test. Three different methods were added: *opening/closing*, a classical method in connecting disconnected areas, *island reconnection* aiming to connect areas identified as islands by attempting to connect them using a minimal-cost path based on grey values, with a defined upper bound on path length to limit computation, and a *structurally aware reconnection* method that used skeletonization to first identify the ends of vessels, and then for each pair of disconnected vessel ends within a defined radius, attempts to reconnect them using a minimal cost path across the probability map.

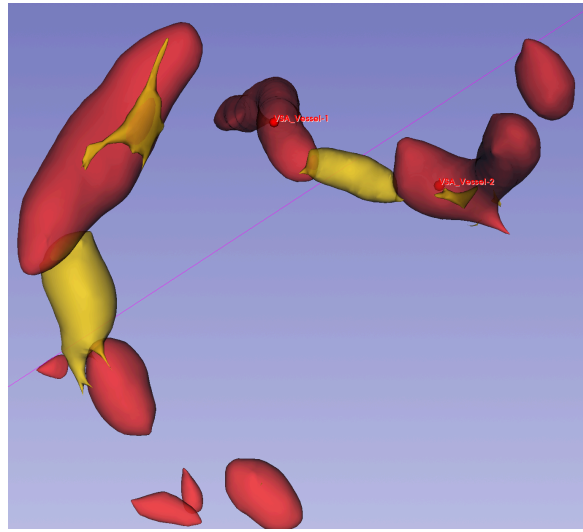


Figure 4.6: Structurally aware reconnection: **Red:** vessels as identified by other steps. **Yellow:** Bridges between tubular endpoints.⁴

4.4.4 Bootstrapping Machine learning

As the user has already provided high confidence starting points in the form of annotations, it is reasonable to assume that high valued points attached to the annotation are vessel. As a result, two methods were used to “expand” user points: the marching cubes algorithm as mentioned in Lesage *et al* [21], and a tube prior algorithm proposed by ITKTubeTK: itktubeTubeExtractor that extracts tubular structures associated with the points. These areas attached to high confidence annotations were then extracted and used to train a small machine learning model: a random forest classifier, with the goal of recovering vessel regions outside the user’s annotation neighborhood.

TODO: add final ML method

⁴3DSlicer smooths visualizations in 3D without combining different classes. The final segmentation here is uniform and continuous.

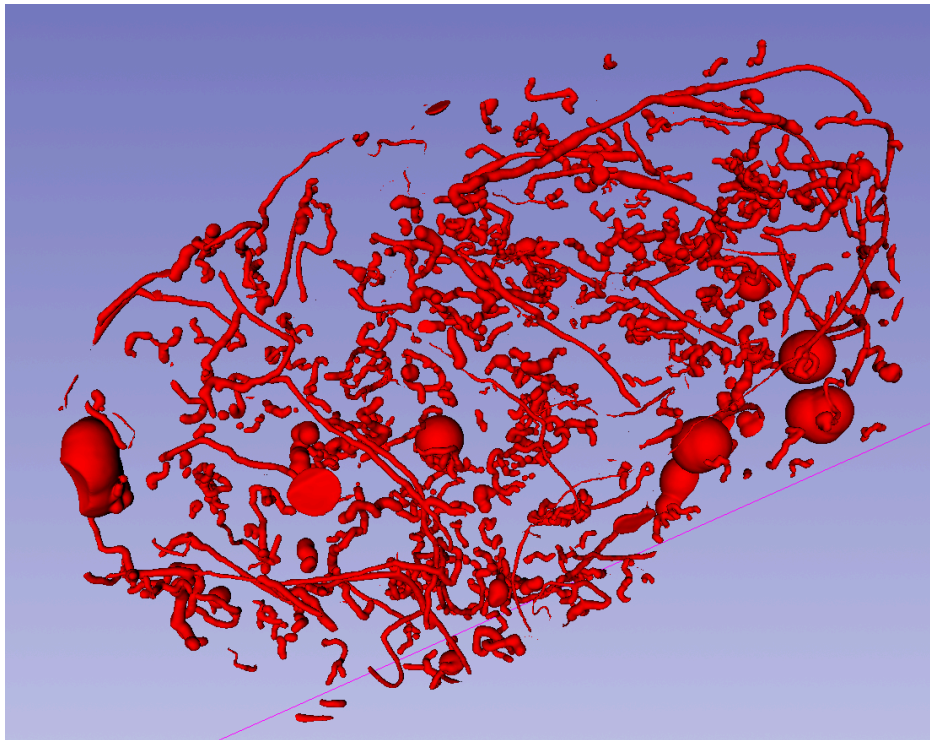


Figure 4.7: Visualization of the segmentation following the multi stage pipeline during development. Successful continuous vessel segmentation and few small noisy sections. Performance, as measured using user annotated points, of 26/27 vessels and 16/16 background points correctly classified. Also visible: Blob-like artifacts from flood fill weight not being suppressed by Frangi, alongside disconnections in due to larger low signal areas.

4.5 Ablation study: measuring component contribution

It is common for software approaches comprising multiple steps to not correctly quantify the contribution of each individual step to the overall algorithm performance, as well as its impact on resource utilization and computation time. Initial plugin development used the CA-LL-R dataset, the smallest uncompressed dataset Figure 4.1, to enable rapid prototyping and evaluation of the performance. Total memory usage was high, approaching the limits of the device used for testing and development, and inference time reaching into the hours.

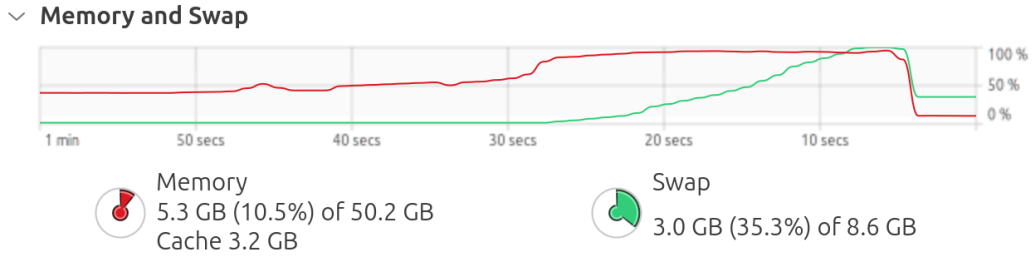


Figure 4.8: RAM utilization before & during segmentation: baseline after loading dataset, below 24GB utilization. During processing 100% of the RAM and swap are used, meaning that this dataset already represented the approximate upper bound to dataset size able to be processed at this point.

As a result, to properly understand the performance/cost tradeoff of the steps, and enable the expansion of the testing to the larger datasets, it was decided to perform an ablation study. In order to ablate the algorithm, the weights of the probability map of each step was set to 0 except the one under test, which was done for all the parameters of the pipeline. For performance measurements, RAM and CPU utilization were monitored alongside runtime with unused steps disabled. This showed that the Frangi vesselness step was the most important to obtaining high vessel extraction performance, offering the same 26/27 correctly classified vessel points as the full pipeline. Additionally the performance evaluation method was highlighted as lacking: false positives and discontinuities in the blood vessels were not properly penalized, and small vessels were not being correctly segmented; all things that point wise annotations fail to capture.

Method	CPU Threads	RAM GB	RAM Peak GB	Performance Contribution	Inference Time (s)
Baseline	1	9	NA	NA	0
Loaded dataset & point annotation	1	13	NA	NA	0
Pre-processing (shell, thresholding)	1	13.4	19	24 of 27	108
Vesselness	20	22.5	38.4	26 of 27	179
Gap bridging	1	28	28	Minor	225
Tree training and inference	1	16	36.2	Minor	5973

Table 4.2: Ablation study measurements of principal steps on CA-LL-R with development machine Appendix A.5. Performance measured in correctly classified vessel points.

TODO: revisit this with more detail and update figures.

4.5.1 Findings

The removal of *shell removal* resulted in improved performance for extraction of vessels in the outer shell, as noted previously being a crucial point. *Gap bridging* was not identified as being a substantial performance consumer, and did not influence performance metrics but, as it operates on areas that are lacking annotations, it was not possible to know if its contribution was relevant using user annotated points. *Random forest* removal resulted in a large performance gain: vessel extraction went from taking multiple hours to under 20 minutes. It also successfully reduced RAM and swap usage, enabling effective running of the algorithm on larger samples. Additionally, the *probability map* itself was investigated: when every step generates a 3D new volume to hold the probabilities, RAM use naturally increases. The removal of steps thus directly reduces RAM use, and subsequently it was decided to unify the probability accumulation into a single, shared map for all steps.

As a result of this ablation study, the shell removal was removed, the probability map was condensed, and machine learning was moved to an external optional step, with focus for the final algorithm on use of a simple, scientifically grounded extraction based on frangi with a step for combatting the disconnections in vessels, and an improvement in the performance measurement method in the form of manually annotated data.

4.6 A simplified, scalable pipeline

The final pipeline detailed here with examples

Chapter 5

Results

5.1 Data annotation for evaluation

As mentioned, the user is expected to place vessel, background and outside-of-volume points. These act as points used to define hyperparameters of the algorithms, but also as a performance metric: when the pipeline is run, feedback is given with how many vessel points are correctly classified. However this method of performance evaluation has shortcomings: it evaluates the data in a pointwise fashion, ignoring critical elements for downstream tasks such as connectivity, and relies on the human evaluating a 2D plane, ignoring parameters such as gap filling. As users also place points generally towards the center of the vessels, there is little measurement of the width of vessels beyond if a background point ends up being caught in the vessel prediction.

As a result, it was decided to provide more dense annotations in the form of fully annotated regions taken from different scans: a script was written to subsample large scans into 8x8x8 regions, from which, for each scan, three subregions were selected, one at each distance step from the center. This method was chosen as it guarantees uniform scaling, easily loadable and feasibly annotatable scans, and offers a good perspective on algorithm performance.

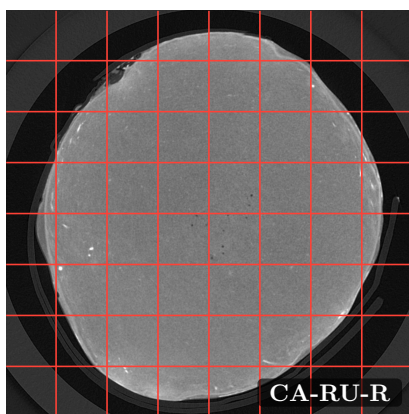


Figure 5.1: Grid subsample of a tumor for annotation. TODO: make this clearer and detail data distribution

5.2 Performance results

Using the loss functions from 3D Slicer:

Using our own script on the 3D Slicer exported data: discuss the performance using DICE, cLDICE, and skeletonization

Chapter 6

Conclusion and future perspectives

The work laid out in this thesis set out to bring computer science techniques etc

Appendices

Appendix A

Appendices

1.1 List of software for working with 3D data

Software	License model	Scriptable
Commercial / closed source		
Amira	Proprietary, paid	Yes
Analyze	Proprietary, paid	Limited
Aphelion	Proprietary, paid	Yes
Avizo	Proprietary, paid	Yes
CTAn / CTVol	Proprietary, paid (Bruker)	Limited
Dragonfly3D	Proprietary; FreED (free, non-commercial)	Yes (Python)
Image-Pro	Proprietary, paid	Yes
Imaris	Proprietary, paid	Yes
MeVisLab	Proprietary; free academic	Yes
Mimics	Proprietary, paid	Yes
Octopus	Proprietary, paid	Limited
ScanIP	Proprietary, paid	Yes
VG Studio	Proprietary, paid	Yes
Zeiss Inspect	Proprietary, paid	Limited

Software	License model	Scriptable
Open source / freeware		
3D Slicer	BSD (modified)	Yes (Python, C++)
Blender	GPLv3	Yes (Python)
Chimera / ChimeraX	Free academic	Yes (Python)
Dream3D	BSD	Yes
FIJI / ImageJ	GPLv2 / Public domain	Yes (Java, macros)
IMOD	GPLv2	Yes
MeshLab	GPLv3	Yes
OsiriX (Lite)	LGPL (Lite); proprietary (MD)	Limited
ParaView	BSD	Yes (Python)
SimVascular	BSD	Yes
VesselKnife	Open source (research)	Yes
VisIt	BSD	Yes (Python)

Table 1.4: Landscape of 3D imaging analysis software relevant to micro-CT vascular work, grouped by licensing model. *Scriptable* indicates if an extension interface exists.

1.2 CECT dataset performance challenges (TODO: revisit, this was excised from the main text)

Data management for Micro-CT scans is a challenge for users: after a scan is completed, they receive data from the CT machine in the form of a collection of 16 bit TIFF files: heavy, with a single 2000x2300 slice at 16bits per pixel weighing **9.2MB**, or as is often the case the data is saved as 3 channels, resulting in 27.6MB, and a whole 2400 slice scan weighing at least **22.1GB**. Scans are then windowed to 8 bit, occasionally with some form of compression, and the empty slices are removed: this generally halves or more the total data amount. This windowing process was documented as being unprincipled: the window was chosen based on the researchers best judgment, and the original uncompressed data discarded.

Furthermore, certain researchers would then carry out a lossy compression of the data in the form of JPEG image slices, as was the case with the data used in this thesis: the total scan weights provided ranged from **0.103** to **13.2GB** (597x698x854 to 3000x3000x2653) and the original lossless data was not preserved, in both cases the windowing and the compression were motivated by data storage cost concerns.

Finally, the provided data was generally given with little or no context: the data was provided in the form of a folder containing images as well as experiments that were run, with no associated dates and without grounding context such as the scan voxel size or parameters of the scanning machine. These issues of dataset size and compression resulted in challenges unforeseen during the literature review which required particular attention.

The total data required for an uncompressed scan can reach into the tens or hundreds of GB. During the initial software evaluation, 3D Slicer was successful in loading all datasets on the development machine - however it was not verified at the time how much memory was being used. The testing of the pipeline on other datasets revealed the performance limitations of the implemented approach: with initial end to end runtime being about an hour and requiring 24GB of system memory, larger datasets saw an increase in inference time to un-manageable levels, as well as limitations of system memory.

These performance issues have multiple sources: when implementing a 3D Slicer plugin in python, a single thread is available, and this thread locks all other 3D Slicer activity (this fact extends to other 3D Slicer functions such as loading and saving). When running on a large scan, combined with the generation of probability maps and the sequential algorithms, memory usage exceeded ram, reached into swap, and could run seemingly indefinitely (success was only observed on smaller scans). This is a known issue with 3D Slicer⁵.

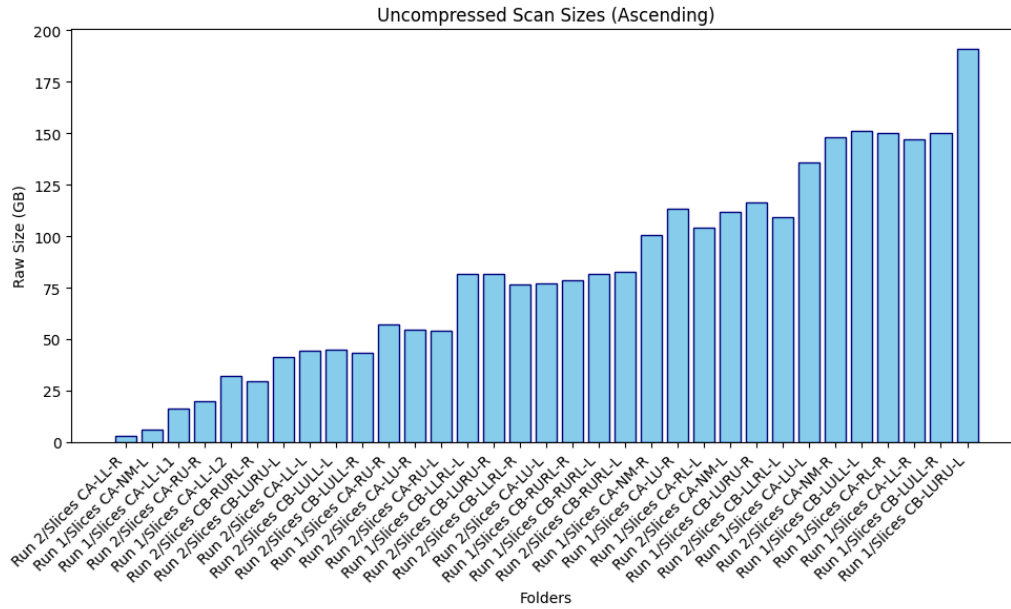


Figure 1.1: TODO: This is a placeholder. TODO: Add horizontal lines for the RAM of computers.

Visualization of the raw dataset sizes, obtained by multiplying width, height and depth by 8 bits per pixel

RAM use: originally discussed in the methodology

⁵Performance limitations as discussed on here the forums

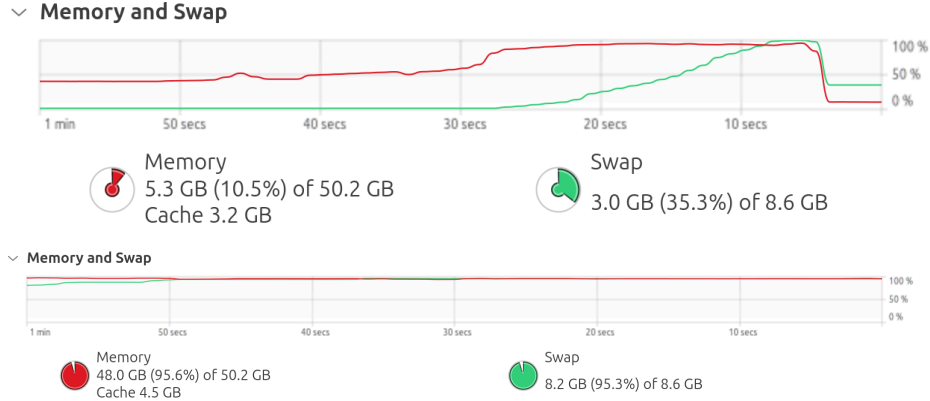


Figure 1.2: **Top:** CA-LL-L2 RAM utilization during segmentation: baseline after loading dataset, below 24GB utilization, during processing 100% RAM and swap are used, **Bottom:** CB-LURU-R showing full RAM and SWAP utilization when loaded (no inference)

1.3 Environmental and CO2 impact of the thesis (TODO: revisit)

The main sources of CO2 impact of this thesis are the use of computational resources and the human factor.

1. During the thesis, the laptop of the writer was used: A legion 7i slim with 48GB DDR5, i7-13700H, 8GB RTX4070.
 1. **Production CO2:** The compliance document provided by Lenovo details an estimated carbon footprint of 429g +/- 86g CO2e. For a lifetime of 8 years, and an estimated thesis length of 3 months equivalent full time, this equates to **13.41Kg CO2e**.
 2. **Usage CO2:** The total estimated time dedicated to the coding and writing of the thesis is estimated based on the credits and approximate time investment per credit: 25 credits at 30 hours per credit equates to 750 hours. The laptop was measured over one 2h coding + writing session as using 40.89Wh, equating to a continuous use of 20.45w. This works out to 15.34 kWh, with an estimated Belgian CO2/kWh by the AIB (Association of Issuing Bodies) in 2024 of 131.73 g/kWh, resulting in total emissions of **2.02Kg CO2**.
 3. **AI Utilization:** The impact of LLMs from large providers such as OpenAI is an ongoing research topic and difficult to measure, on top of which are layered issues like the use of Piccolo, UCLouvains aggregation service. A total of TODO credits were used on Piccolo, with an unknown impact. Claude Code was also used, with a total of 3 separate conversations containing TODO 41 cumulative prompts. The impact of AI use is thus left out of the estimation, with a guesstimate below.
2. The thesis required meetings, for which the main source of CO2 emissions is the travel to/from Louvain-La-Neuve. This travel was done primarily by train, and the amount of travel directly attributable to the thesis was of 21 round trips, with a distance per trip of 93.2Km at an estimated 16.6g CO2e/Km, equating to **32.49Kg CO2e**.

There was minimal extra data storage required to complete this thesis, however it is important to note that the large files handled do incur emissions if duplicated and stored across devices outside the user's own computer.

The prior calculations equate to a total approximate impact of **47.92 Kg CO₂e**.

As for AI - a rough impact is estimated using the computer of the writer as a basis: three of the prompts sent to Piccolo and Claude were sent to the largest local qwen instance fitting in the 8GB GPU, and the time to answer as well as power use while answering estimated: for the three prompts, a total of TODO minutes to answer was required, with a power use of TODO

1.4 Use of LLMs and AIs

The main two AI assistants used in this thesis were Piccolo, offered by UCLouvain and used for writing feedback, and ClaudeCode, by Anthropic, for the expansion and creation of the plugin.

1.5 Development computer specifications

Development computer:

Lenovo legion 7i slim, 48GB DDR5, i7-13700H, 8GB RTX4070

laboratory computer provided to students:

TODO: i7, 32GB, 3060ti ??

Appendix B

Extra figures (TODO: revisit)

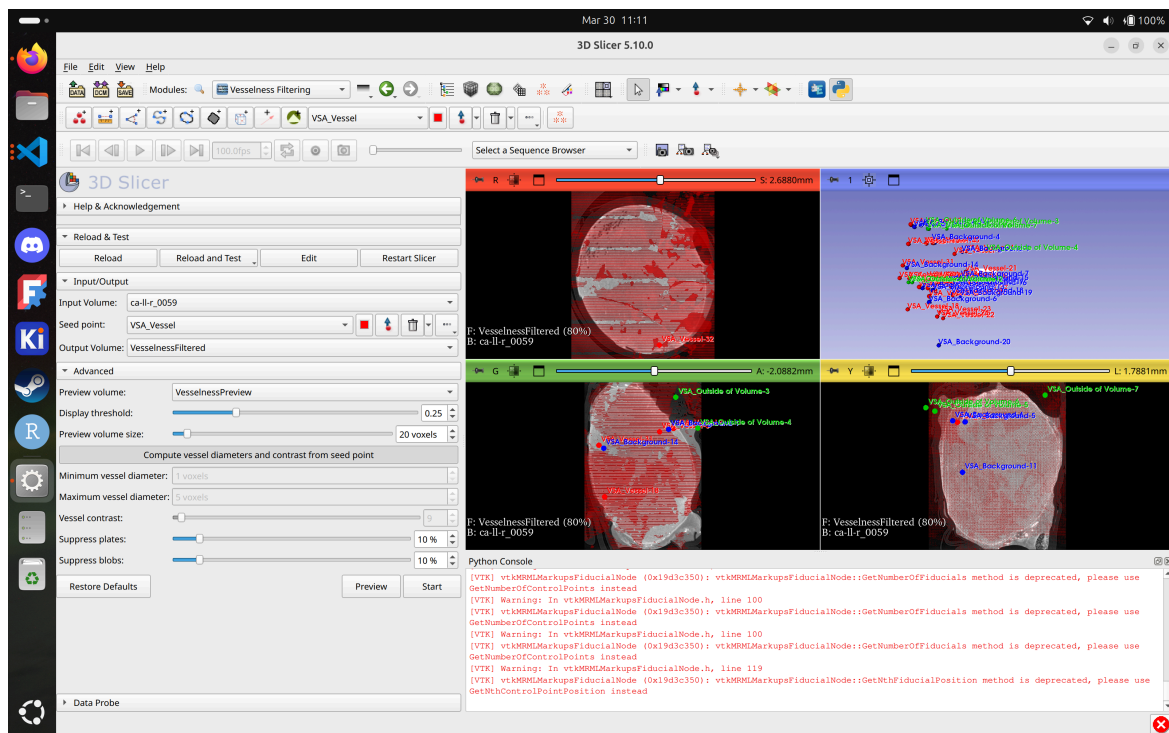


Figure 2.1: Demonstration of one of the failing plugins and the lack of user feedback on what went wrong

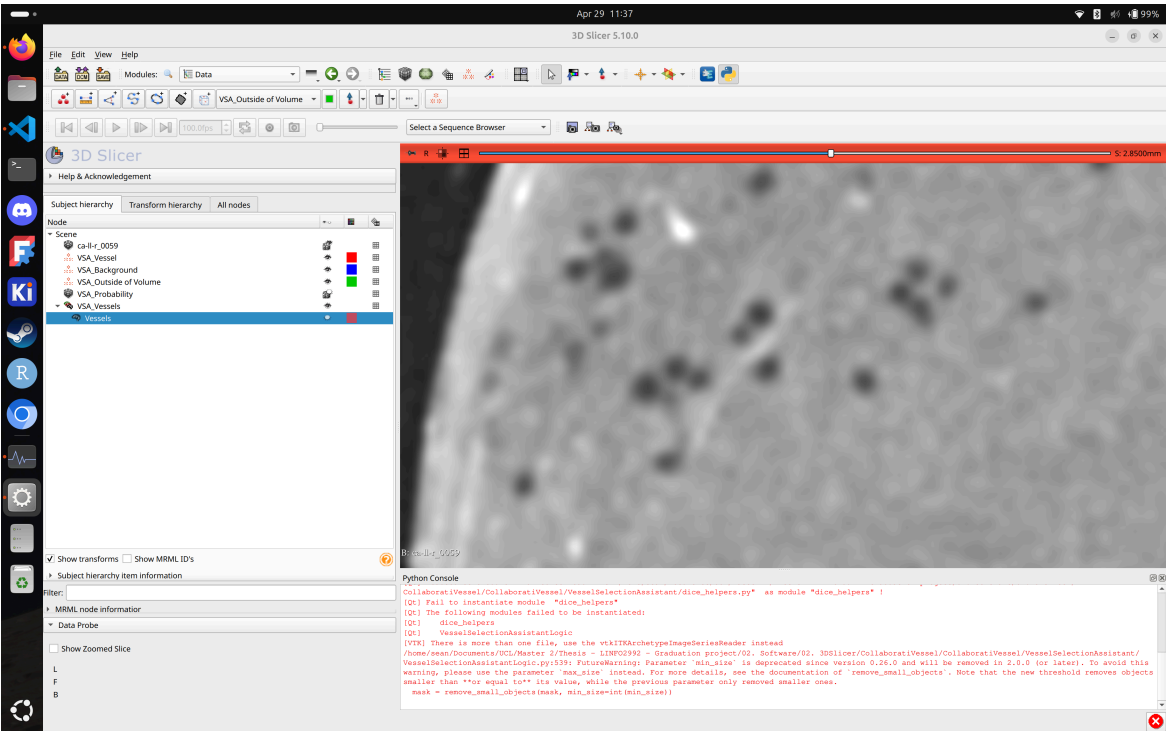


Figure 2.2: Areas of CA-LL-R where vessels are detected because of the black holes around it

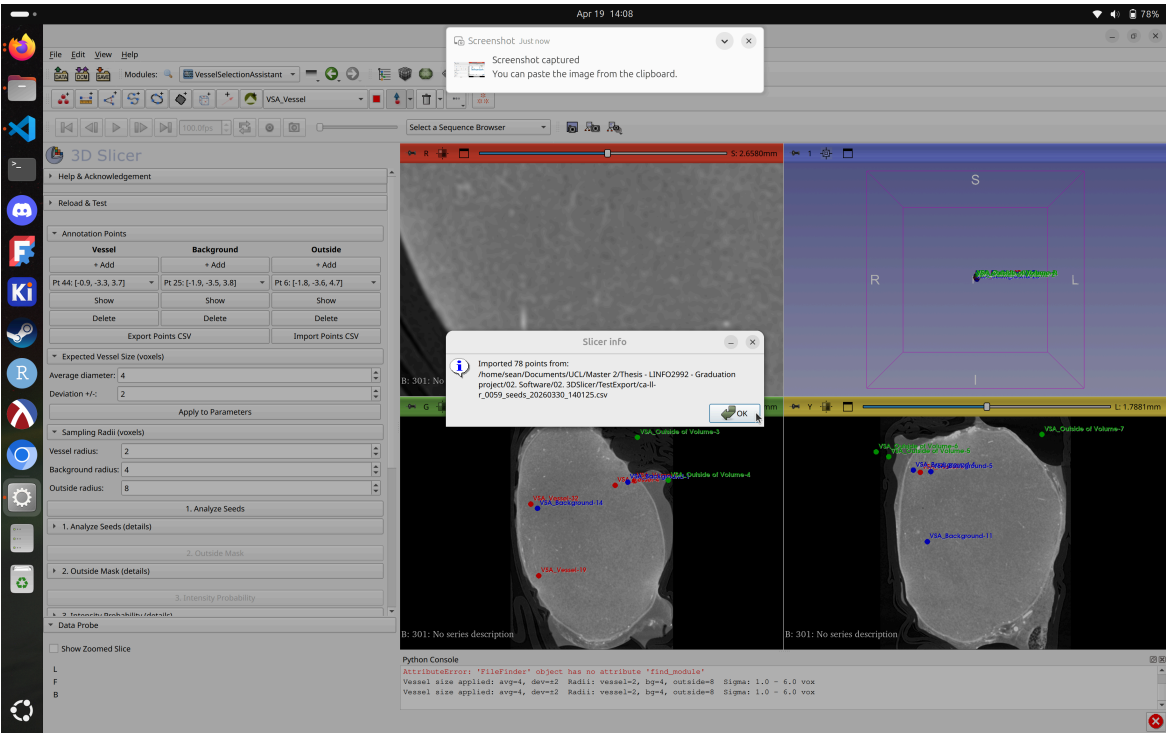
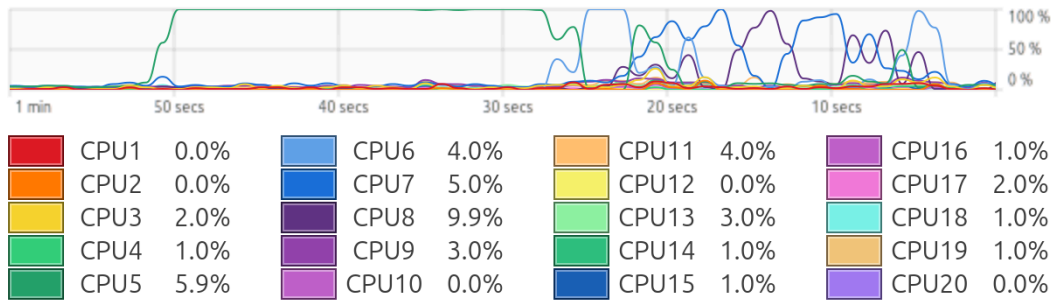


Figure 2.3: Import process

✓ CPU



✓ Memory and Swap

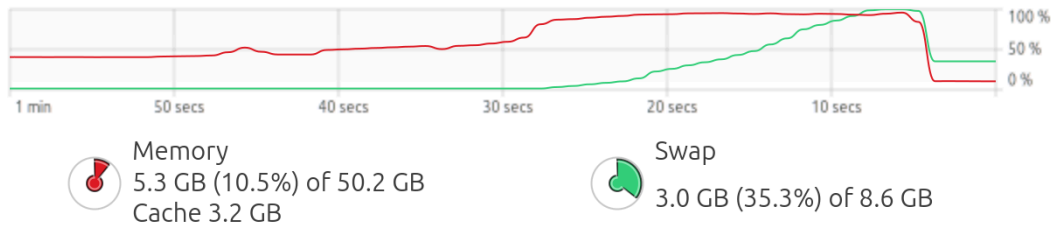


Figure 2.4: RAM and CPU use during a run of the initial pipeline on CA-LL-R showing the efficiency issues that Python and 3D Slicer cause

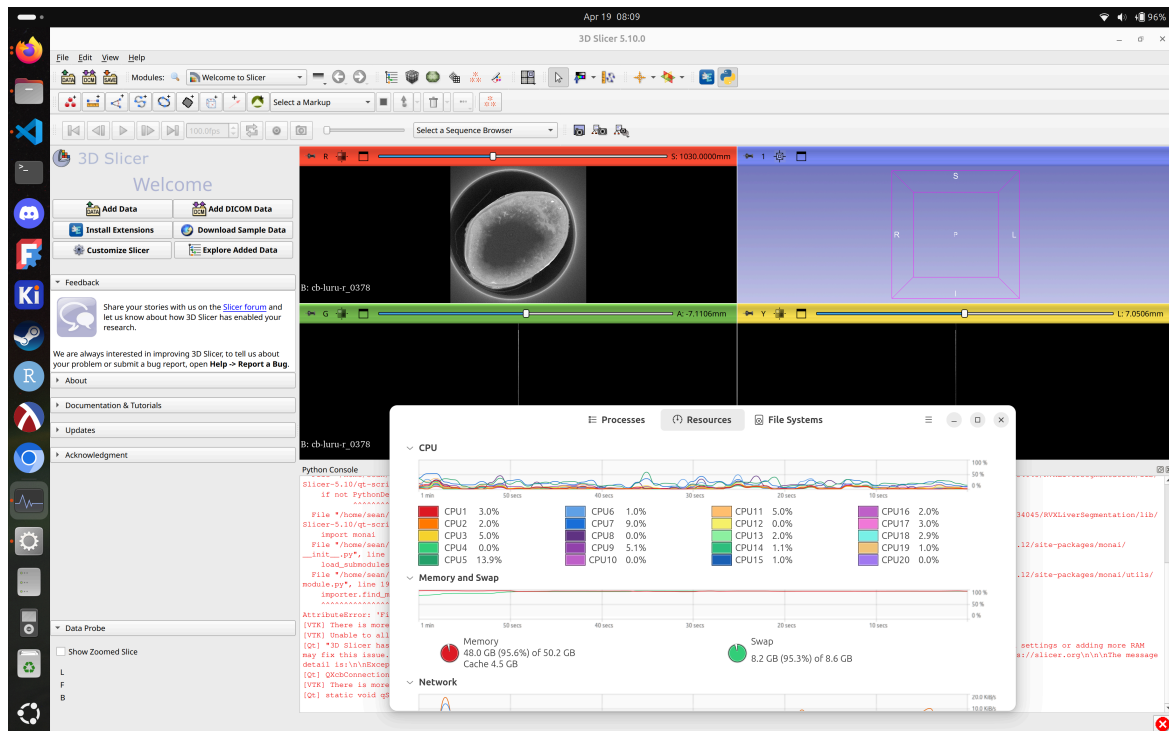


Figure 2.5: RAM use loading a large dataset, showcasing the issue with 3D Slicer's method of loading data into memory

Bibliography

- [1] A. F. Frangi, W. J. Niessen, K. L. Vincken, and M. A. Viergever, “Multiscale vessel enhancement filtering,” in *Medical Image Computing and Computer-Assisted Intervention — MICCAI’98*, W. M. Wells, A. Colchester, and S. Delp, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 130–137.
- [2] L. for Percutaneous Surgery, “Tutorial on 3D Slicer python scripting and programming.” [Online]. Available: <https://github.com/Slicer/SlicerProgrammingTutorial>
- [3] T. Balcaen *et al.*, “Revealing the Three-Dimensional Murine Brain Microstructure by Contrast-Enhanced Computed Tomography,” *Frontiers in Neuroscience*, vol. 17, p. 1141615, 2023, doi: [10.3389/fnins.2023.1141615](https://doi.org/10.3389/fnins.2023.1141615).
- [4] J. Pichat, J. E. Iglesias, T. Yousry, S. Ourselin, and M. Modat, “A Survey of Methods for 3D Histology Reconstruction,” *Medical Image Analysis*, vol. 46, pp. 73–105, 2018, doi: [10.1016/j.media.2018.02.004](https://doi.org/10.1016/j.media.2018.02.004).
- [5] S. de Bournonville, S. Vangrunderbeeck, and G. Kerckhofs, “Contrast-Enhanced MicroCT for Virtual 3D Anatomical Pathology of Biological Tissues: A Literature Review,” *Contrast Media and Molecular Imaging*, vol. 2019, p. 8617406, 2019, doi: [10.1155/2019/8617406](https://doi.org/10.1155/2019/8617406).
- [6] G. S. G. S. J. H. N. C. C. B. S. E. H. B. M. Anna K. Hans D., “Extending two-dimensional histology into the third dimension through conventional micro computed tomography,” *NeuroImage*, vol. 139, pp. 26–36, 2016, doi: [http://dx.doi.org/10.1016/j.neuroimage.2016.06.005](https://doi.org/http://dx.doi.org/10.1016/j.neuroimage.2016.06.005).
- [7] D. Lin *et al.*, “\textit{E pluribus unum}: prospective acceptability benchmarking from the Contouring Collaborative for Consensus in Radiation Oncology crowdsourced initiative for multiobserver segmentation,” *Journal of Medical Imaging*, vol. 10, no. S1, p. S11903, Feb. 2023, doi: [10.1117/1.JMI.10.S1.S11903](https://doi.org/10.1117/1.JMI.10.S1.S11903).
- [8] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation.” [Online]. Available: <https://arxiv.org/abs/1505.04597>
- [9] C. T. C. Inc., “Dragonfly FreeD License Model.” [Online]. Available: <https://dragonfly.comet.tech/en/products/license-models/freed-license>
- [10] G. Tetteh *et al.*, “DeepVesselNet: Vessel Segmentation, Centerline Prediction, and Bifurcation Detection in 3-D Angiographic Volumes,” *Frontiers in Neuroscience*, vol. 14, p. 592352, 2020, doi: [10.3389/fnins.2020.592352](https://doi.org/10.3389/fnins.2020.592352).
- [11] Y. He *et al.*, “SPROUT: A User-friendly, Scalable Toolkit for Multi-class Segmentation of Volumetric Images,” *bioRxiv*, 2024, doi: [10.1101/2024.11.22.624847](https://doi.org/10.1101/2024.11.22.624847).
- [12] I. O. team, “Frangi Vesselness – ImageJ.”

- [13] K.-C. J. F. J. F.-R. J.-C. P. S. B. C. J. D. F. F. S. M. B. J. A. S. M. J. P. S. K. R. Fedorov A. Beichel R., “3D Slicer as an Image Computing Platform for the Quantitative Imaging Network,” *Magnetic Resonance Imaging*, vol. 30, no. (9), pp. 1323–1341, Nov. 2012, doi: [10.1016/j.mri.2012.05.001](https://doi.org/10.1016/j.mri.2012.05.001).
- [14] R. Izzo, D. Steinman, S. Manini, and L. Antiga, “The Vascular Modeling Toolkit: A Python Library for the Analysis of Tubular Structures in Medical Images,” vol. 3, no. 25. p. 745, 2018. doi: [10.21105/joss.00745](https://doi.org/10.21105/joss.00745).
- [15] A. Affane *et al.*, “The R-Vessel-X Project,” *arXiv preprint*, 2025.
- [16] L. He, L. R. Long, S. Antani, and G. R. Thoma, “Histology Image Analysis for Carcinoma Detection and Grading,” *Computer Methods and Programs in Biomedicine*, vol. 107, no. 3, pp. 538–556, Sept. 2012, doi: [10.1016/j.cmpb.2011.12.007](https://doi.org/10.1016/j.cmpb.2011.12.007).
- [17] P. B. N. M. B. C. M. A. K. & F. P. Juliana Martins de S. e Silva Irene Zanette, “Three-dimensional non-destructive soft-tissue visualization with X-ray staining micro-tomography,” 2015.
- [18] A. Maes *et al.*, “Cryogenic contrast-enhanced microCT enables nondestructive 3D quantitative histopathology of soft biological tissues,” *Nature Communications*, vol. 13, no. 1, p. 6207, Oct. 2022, doi: [10.1038/s41467-022-34048-4](https://doi.org/10.1038/s41467-022-34048-4).
- [19] Z.-M. S. D. C. Ko-Chin Chen Alon Arad, “High-definition neural visualization of rodent brain using micro-CT scanning and non-local-means processing,” *BMC Med. Imaging*, vol. 18, p. 38, Oct. 2018, doi: [10.1186/s12880-018-0280-6](https://doi.org/10.1186/s12880-018-0280-6).
- [20] G. K. Lisa Leyssens Camille Pestiaux, “A Review of Ex Vivo X-ray Microfocus Computed Tomography-Based Characterization of the Cardiovascular System,” 2021.
- [21] D. Lesage, E. D. Angelini, I. Bloch, and G. Funka-Lea, “A review of 3D vessel lumen segmentation techniques: Models, features and extraction schemes,” *Medical Image Analysis*, vol. 13, no. 6, pp. 819–845, 2009, doi: <https://doi.org/10.1016/j.media.2009.07.011>.
- [22] F. G. S. B. D. M. J. M. S.-A. R. H. G. M. N. R. K. W. L. F. K. T. L. Josef Ehling Benjamin Theek, “Micro-CT Imaging of Tumor Angiogenesis Quantitative Measures Describing Micromorphology and Vascularization,” *Am. J. Pathol.*, vol. 184, pp. 431–441, Feb. 2014, doi: [10.1016/j.ajpath.2013.10.014](https://doi.org/10.1016/j.ajpath.2013.10.014).
- [23] N. Otsu, “A Threshold Selection Method from Gray-Level Histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979, doi: [10.1109/TSMC.1979.4310076](https://doi.org/10.1109/TSMC.1979.4310076).
- [24] M. Mancas, B. Gosselin, and B. Macq, “Segmentation using a region-growing thresholding,” in *Image Processing: Algorithms and Systems IV*, E. R. Dougherty, J. T. Astola, and K. O. Egiazarian, Eds., SPIE, 2005, pp. 388–398. doi: [10.1117/12.587995](https://doi.org/10.1117/12.587995).
- [25] P. Soille and L. Vincent, “Watersheds in Digital Spaces: An Efficient Algorithm Based on Immersion Simulations,” *IEEE Transactions on Pattern Analysis & Machine Intelligence*, vol. 13, no. 6, pp. 583–598, June 1991, doi: [10.1109/34.87344](https://doi.org/10.1109/34.87344).
- [26] Y. Sato *et al.*, “Three-dimensional multi-scale line filter for segmentation and visualization of curvilinear structures in medical images,” *Medical Image Analysis*, vol. 2, no. 2, pp. 143–168, 1998, doi: [https://doi.org/10.1016/S1361-8415\(98\)80009-1](https://doi.org/10.1016/S1361-8415(98)80009-1).
- [27] “Beyond Frangi: An improved multiscale vesselness filter,” 2023, doi: [10.1117/12.2081147](https://doi.org/10.1117/12.2081147).
- [28] J. Lamy, O. Merveille, B. Kerautret, and N. Passat, “A Benchmark Framework for Multiregion Analysis of Vesselness Filters,” *IEEE Transactions on Medical Imaging*, vol. 41, no. 12, pp. 3649–3662, 2022, doi: [10.1109/TMI.2022.3192679](https://doi.org/10.1109/TMI.2022.3192679).

-
- [29] “Tubular Structure Segmentation Based on Minimal Path Method and Anisotropic Enhancement,” 2011, doi: [10.1007/s11263-010-0331-0](https://doi.org/10.1007/s11263-010-0331-0).
 - [30] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *CoRR*, 2016, [Online]. Available: <http://arxiv.org/abs/1603.02754>
 - [31] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: [10.1145/3065386](https://doi.org/10.1145/3065386).
 - [32] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger, “3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation.” [Online]. Available: <https://arxiv.org/abs/1606.06650>
 - [33] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-End Object Detection with Transformers,” *CoRR*, 2020, [Online]. Available: <https://arxiv.org/abs/2005.12872>
 - [34] A. Kirillov *et al.*, “Segment Anything.” [Online]. Available: <https://arxiv.org/abs/2304.02643>
 - [35] S. A. A. K. J. P. & K. H. M.-H. Fabian Isensee Paul F. Jaeger, “nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation,” *Nature Methods*, vol. 18, pp. 203–211, 2021, doi: [10.1038/s41592-020-01008-z](https://doi.org/10.1038/s41592-020-01008-z).
 - [36] F. Semeraro, A. M. Quintart, S. F. Izquierdo, and J. C. Ferguson, “TomoSAM: A 3D Slicer extension using SAM for tomography segmentation,” *SoftwareX*, vol. 31, p. 102218, 2025, doi: <https://doi.org/10.1016/j.softx.2025.102218>.
 - [37] Y.-C. Chen *et al.*, “Coronary Artery Segmentation in Cardiac CT Angiography Using 3D Multi-Channel U-net.” [Online]. Available: <https://arxiv.org/abs/1907.12246>
 - [38] M. Zamir, “Arterial branching within the confines of fractal L-system formalism,” *The Journal of General Physiology*, vol. 118, no. 3, pp. 267–276, Sept. 2001, doi: [10.1085/jgp.118.3.267](https://doi.org/10.1085/jgp.118.3.267).
 - [39] X. Yi, E. Walia, and P. Babyn, “Generative adversarial network in medical imaging: A review,” *Medical Image Analysis*, vol. 58, p. 101552, 2019, doi: <https://doi.org/10.1016/j.media.2019.101552>.
 - [40] F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation.” [Online]. Available: <https://arxiv.org/abs/1606.04797>
 - [41] S. Shit *et al.*, “cLDice - a Topology-Preserving Loss Function for Tubular Structure Segmentation,” *CoRR*, 2020, [Online]. Available: <https://arxiv.org/abs/2003.07311>
 - [42] Y. Zhou *et al.*, “CF-Loss: Clinically-relevant feature optimised loss function for retinal multi-class vessel segmentation and vascular feature measurement,” *Medical Image Analysis*, vol. 93, p. 103098, 2024, doi: <https://doi.org/10.1016/j.media.2024.103098>.
 - [43] “topolosses - Python package providing topology-aware losses for segmentation tasks.” Accessed: Apr. 29, 2026. [Online]. Available: <https://pypi.org/project/topolosses/>
 - [44] Suprosanna and Jocpae, “jocpae/VesselGraph: First public release.” [Online]. Available: <https://zenodo.org/record/5301621>
 - [45] “Initialization, noise, singularities, and scale in height ridge traversal for tubular object centerline extraction,” 2002, doi: [10.1109/42.993126](https://doi.org/10.1109/42.993126).
 - [46] “VesselKnife – software for the analysis of tubular structures in biomedical images,” 2025, doi: [10.1016/j.cmpb.2025.108817](https://doi.org/10.1016/j.cmpb.2025.108817).

- [47] “SimVascular: An Open Source Pipeline for Cardiovascular Simulation,” 2016, doi: [10.1007/s10439-016-1762-8](https://doi.org/10.1007/s10439-016-1762-8).
- [48] 3. D Slicer Community, “Extensions — 3D Slicer documentation.” [Online]. Available: https://slicer.readthedocs.io/en/latest/user_guide/extensions.html#extensions-manager